

Unified AI Trading Operating System and Hermes Protocol

Final Product, Architecture, Engineering, Production, and Codex HITL Build Paper

Version 3.0 | Final Build Baseline

July 10, 2026

FINAL BUILD BASELINE

FULL PRODUCT CAPABILITY | POLICY-DRIVEN OPERATION | CODEX HUMAN-IN-THE-LOOP CONTROL

Source baseline: Architecture v1.0, Product Charter, and Codex HITL standard

Contents

- Executive Summary
- 1. Purpose, Scope, and Build Authority
- 2. Vision, Product Promise, and Design Principles
- 3. Users, Tenancy, and Operating Models
- 4. Product Scope and Capability Catalog
- 5. End-to-End Workflows and Functional Requirements
- 6. Unified System Architecture
- 7. Domain Model and Data Architecture
- 8. API, Event, and Integration Contracts
- 9. Trading Connectivity, Order Management, and Execution
- 10. Risk Management and Quantitative Analytics
- 11. AI and Agent Architecture
- 12. Codex Human-in-the-Loop Engineering Control Plane
- 13. Market Intelligence and Provider Quality
- 14. Hermes Protocol and Multi-Chain Liquidity Intelligence
- 15. User Experience and Interface Architecture
- 16. Security, Identity, Privacy, and Tenant Isolation
- 17. Governance, Compliance, Audit, and Model Oversight
- 18. Reliability, Observability, and Incident Management
- 19. Testing, Evaluation, and Quality Engineering
- 20. DevOps, Environments, Deployment, and Release Management
- 21. Delivery Program, Workstreams, and Phase Plan
- 22. Metrics, Acceptance Criteria, and Readiness Gates
- 23. Final Definition of Done and Build Baseline
- Appendices

Executive Summary

The Unified AI Trading Operating System and Hermes Protocol is a multi-tenant, production-grade platform for trading operations, decision support, quantitative risk management, market intelligence, execution, behavioral coaching, and multi-chain liquidity operations. It combines two product tracks on a common foundation: Track A serves active traders, trading teams, coaches, and risk administrators; Track B serves liquidity operators, protocol teams, and infrastructure administrators who require durable provider intelligence, chain-aware market data, tenant isolation, operational evidence, and incident workflows.

This paper is the final consolidated build baseline. It expands the source architecture and product charter into a complete specification for product scope, domain design, system architecture, APIs, agents, risk controls, execution, security, reliability, deployment, testing, delivery, and production acceptance. It supersedes the earlier index-only charter while preserving the original unified-platform direction, monorepo model, Supabase-backed tenant security, and two-track product strategy.

The build process is governed by a repository-level Codex human-in-the-loop control plane. Specification artifacts, `AGENTS.md`, deterministic TOML policy, explicit approval tokens, validated workflow states, bounded command execution, SQLite local audit, requirement traceability, CI, container verification, and commit-bound evidence are mandatory parts of the final engineering baseline. These controls govern how code and operations are changed; they do not remove or permanently narrow product capabilities.

The build posture is full capability. The codebase shall not impose arbitrary product-scope restrictions on asset class, market, broker, exchange, chain, provider, model vendor, strategy type, account count, data history, automation level, or user workflow. Capabilities that were previously framed as read-only or deferred are included in the architecture as first-class modules. Broker and exchange connections support observation, import, simulation, paper trading, supervised live trading, and policy-authorized automated execution. AI agents may analyze, recommend, prepare actions, or initiate actions when explicitly permitted by tenant policy and role authorization.

Full capability does not mean absence of engineering control. Authentication, tenant isolation, authorization, auditability, transactional integrity, risk evaluation, idempotency, reconciliation, secret protection, and venue or jurisdiction obligations remain mandatory system invariants. All operational limits shall be data-driven, versioned, tenant-scoped, and auditable rather than hard-coded into the product. Authorized administrators can configure policy, escalation, approval, and execution modes without rebuilding the application.

The target architecture uses a TypeScript-first monorepo with Next.js web applications, Node.js APIs, asynchronous workers, Python quantitative and machine-learning workers where appropriate, Supabase Postgres with row-level security, object storage, vector search, cache, event transport, and append-only audit evidence. Integrations are adapter-based so that market-data providers, brokers, exchanges, AI models, chain RPCs, and notification services can be added or replaced without modifying core domain logic.

The operating model is event-driven and evidence-oriented. Commands are validated synchronously, material state changes are committed transactionally, domain events are published through an outbox, asynchronous analysis is performed by workers, and every sensitive action carries tenant, actor, policy, correlation, and provenance metadata. The same evidence chain supports trader review, risk investigation, provider-quality assessment, model evaluation, incident response, and production sign-off.

The finished platform will provide:

- A complete trading journal and performance analytics system with import, normalization, screenshot intelligence, replay, grading, setup analysis, emotional tracking, and coaching.
- A deterministic and configurable risk engine for sizing, exposure, concentration, drawdown, daily loss, leverage, correlation, liquidity, and execution controls.
- Market-open and continuous intelligence using normalized market data, event calendars, news inputs, regime detection, watchlists, and probability analysis.

- A broker and exchange connectivity layer with unified accounts, orders, fills, positions, reconciliation, and multiple execution modes.
- An agent platform with structured outputs, tool permissions, model routing, prompt versioning, retrieval, memory, evaluation, and full execution audit.
- Hermes multi-chain liquidity intelligence with provider registry, endpoint health, quality scoring, quorum decisions, pool and route analytics, anomaly detection, incidents, and operator evidence.
- Enterprise-grade tenancy, RBAC and ABAC, secrets management, encryption, observability, CI/CD, disaster recovery, and production readiness gates.
- A Spec Kit-compatible and Codex-aware implementation system with deterministic policy, approval, state, command, audit, traceability, and verification controls.

The release program remains phased for delivery control, but no phase is permitted to introduce a domain-model dead end. Every phase must extend the same canonical entities, contracts, policies, and evidence model so the system can progress from local development to multi-tenant production without re-platforming.

Purpose, Scope, and Build Authority

Purpose

This paper defines what shall be built, how the platform shall be structured, how its components shall interact, and what evidence is required before the system is considered production-ready. It is intended for product leadership, software engineers, AI and quantitative researchers, security engineers, DevOps and SRE teams, compliance and operations personnel, quality engineers, and implementation partners.

The document is normative. Statements using **shall** are mandatory build requirements. **Should** identifies a strong default that may be changed through an architecture decision record. **May** identifies an optional implementation choice that must remain compatible with the defined contracts.

Source Baseline and Supersession

The paper consolidates two internal source documents dated June 28, 2026:

1. Architecture - AI Trading OS and Hermes Protocol, version 1.0.
2. Product Charter - Unified AI Trading Operating System and Hermes Protocol Production Paper.

It also incorporates the Codex human-in-the-loop starter standard supplied on July 10, 2026, including specification-driven artifacts, `AGENTS.md`, deterministic policy decisions, approval tokens, workflow validation, local audit, safe command execution, traceability, automated tests, CI, containerization, security documentation, operations guidance, and a commit-bound verification report.

It retains the two-track product model, layered architecture, TypeScript and Supabase foundation, monorepo organization, tenant isolation, RBAC, append-only audit, agent orchestration, and production-evidence emphasis. It expands the prior fourteen-phase roadmap and sixteen-workflow catalog into a full system specification.

Where the earlier sources described broker access as read-only and live execution as disabled, this final baseline replaces those blanket limitations with a policy-driven execution architecture. The build includes read-only and live capabilities; deployment policy determines which modes are exposed to a tenant, role, account, strategy, or environment.

Full-Capability Build Doctrine

The following rules define the requested no-arbitrary-restrictions posture:

- The domain model shall support all listed product capabilities from the first production schema, even when a user interface is delivered later.

- Feature availability, order permissions, risk thresholds, provider selection, data retention, model selection, and approval requirements shall be controlled by versioned policy records rather than hard-coded branches.
- The integration layer shall be vendor-neutral. No broker, exchange, model provider, cloud provider, market-data vendor, or chain RPC shall be treated as the only supported implementation.
- The platform shall support equities, options, futures, foreign exchange, digital assets, tokenized assets, and additional instrument types through extensible instrument metadata and venue adapters.
- The execution stack shall support simulation, paper, manual live, supervised automation, and autonomous automation. Each mode shall use the same order, risk, audit, and reconciliation contracts.
- The platform shall not impose architecture-level caps on tenants, users, accounts, strategies, instruments, chains, providers, or historical records. Capacity limits shall be operational configuration tied to infrastructure and commercial policy.
- AI agents shall not be restricted to prose generation. They may call authorized tools, create structured artifacts, open workflows, prepare orders, and initiate policy-permitted commands.
- All material restrictions required by an external venue, data license, law, contract, or tenant policy shall be represented explicitly, with provenance and effective dates.
- Engineering-agent controls shall constrain how changes and commands are authorized and evidenced, but shall not be used to delete capabilities from the product architecture or to convert temporary delivery sequencing into permanent scope limitations.

Mandatory System Invariants

The full-capability doctrine does not permit the build to remove controls that preserve system correctness. The following invariants apply in every environment:

- Every request, record, event, job, and artifact is associated with a tenant or an explicitly global system scope.
- Every sensitive mutation is authorized, validated, correlated, and audited.
- Commands that can create financial or operational exposure are idempotent and reconciled against the system of record.
- Secrets are never stored in plaintext application tables or emitted to logs, traces, prompts, analytics, or client bundles.
- Risk and execution decisions are reproducible from versioned inputs, policies, market snapshots, and code versions.
- AI-generated structured outputs are schema-validated before they can reach a domain command handler.
- Provider disagreement, stale data, partial failure, and chain reorganization are represented as explicit states rather than silently ignored.
- Production readiness is evidence-based and cannot be declared solely by manual assertion.
- Repository-changing agent actions use deterministic policy, exact request fingerprints, bounded execution, and commit-bound audit evidence; generative output alone cannot authorize a command.

Document Control

Item	Value
Document	Unified AI Trading Operating System and Hermes Protocol - Final Production Paper
Version	3.0
Status	Final build baseline
Effective date	July 10, 2026
Supersedes	Architecture v1.0, index-level Product Charter dated June 28, 2026, and Final Paper v2.0
Primary stack	Next.js, TypeScript, Node.js, Python workers, Supabase Postgres, object storage, vector search, cache, event transport

Item	Value
Build posture	Full product capability; policy-driven operation; Codex HITL engineering controls; no arbitrary architecture-level feature restrictions
Change authority	Approved architecture decision record plus product, engineering, security, and operations review

Vision, Product Promise, and Design Principles

Vision

The platform is a unified operating system for disciplined trading and liquidity operations. It turns fragmented records, market data, broker activity, on-chain observations, strategy rules, risk policy, behavioral context, and AI analysis into one coherent command environment. It is not only a dashboard or chatbot; it is a durable operational system with canonical data, repeatable workflows, controlled automation, and measurable outcomes.

Core Product Promise

The product shall help traders and market operators improve decision quality, protect capital, detect drift, understand conditions, execute repeatable processes, and learn from evidence. The system shall reduce the distance between observation and action while preserving a complete record of what data was available, what policy applied, what the system recommended, what the user or agent did, and what outcome followed.

Strategic Outcomes

The build is designed to produce six strategic outcomes:

1. **Operational coherence.** Accounts, trades, strategies, market context, risk, coaching, execution, liquidity data, and incidents share one identity and evidence model.
2. **Measurable improvement.** The platform measures process quality and decision quality, not only profit and loss.
3. **Configurable control.** Teams can express risk and operating policy without changing application code.
4. **Provider resilience.** Market and chain data are evaluated for freshness, divergence, completeness, latency, and reliability before use.
5. **Safe automation at full scope.** Automation can progress from recommendations to autonomous execution without replacing core infrastructure.
6. **Production evidence.** Security, quality, model behavior, deployment, recovery, and tenant isolation are demonstrated through repeatable tests and artifacts.

Product Design Principles

- **Canonical data before intelligence.** AI and analytics operate on normalized, reconciled domain records.
- **Deterministic core, probabilistic assistance.** Financial calculations, policy checks, state transitions, and accounting use deterministic services; models contribute interpretation, classification, summarization, forecasting, and workflow assistance.
- **Policy over hard-coded limitation.** Operational behavior is expressed in versioned rules and authorization rather than hidden code branches.
- **Explain every material decision.** Risk results, provider quorum, agent conclusions, and execution actions retain inputs, rule identifiers, confidence, and provenance.
- **One contract across modes.** Simulation, paper, and live pathways share schemas and state machines so test evidence transfers to production.
- **Adapter isolation.** External systems are accessed through typed ports and adapters with contract tests and circuit breakers.

- **Tenant-first security.** Tenant context is resolved once and enforced in database, API, worker, storage, event, and analytics layers.
- **Progressive disclosure.** Interfaces serve novice, professional, and operator users without hiding the underlying evidence.
- **Observable by default.** Logs, metrics, traces, agent runs, policy decisions, and audit events are part of feature completion.
- **No silent degradation.** Staleness, partial data, fallback models, missing providers, or uncertain conclusions must be visible.

Product Boundaries

The platform is an engineering and operating system. It can support decision assistance and automated execution, but it does not replace venue rules, broker controls, contractual data rights, tax advice, or jurisdiction-specific legal review. Those obligations are represented through policy, evidence, and deployment configuration rather than used as a reason to omit capabilities from the build.

Users, Tenancy, and Operating Models

Primary Personas

Persona	Primary objectives	Core capabilities
Individual trader	Improve consistency, risk discipline, and review quality	Journal, risk, briefing, coach, replay, analytics, execution
Trading team member	Execute assigned strategies within team policy	Shared playbooks, limits, approvals, watchlists, execution
Coach or mentor	Review behavior, setups, and process adherence	Cohort analytics, comments, review queues, coaching plans
Risk administrator	Define and monitor limits and exceptions	Policy editor, real-time checks, breach queue, approvals, kill switches
Portfolio manager	Allocate capital and supervise strategies	Portfolio exposure, scenario analysis, strategy comparison, execution
Hermes liquidity analyst	Assess multi-chain liquidity and route quality	Pool, depth, provider, quorum, anomaly, and route intelligence
Hermes operator	Maintain providers, incidents, and evidence	Provider registry, health, incidents, runbooks, audit exports
Compliance reviewer	Review decisions, records, and changes	Audit search, policy history, model evidence, approvals, exports
Tenant administrator	Manage membership, credentials, policy, and integrations	RBAC, accounts, connectors, secrets, environment settings
Platform administrator	Operate the SaaS and shared infrastructure	Global health, tenant support, billing hooks, deployment controls
Developer or integration partner	Extend data, execution, or workflow support	SDKs, APIs, webhooks, sandbox, contract tests, adapter framework

Tenant Models

The tenancy model shall support:

- A single-user tenant for an individual trader.
- A multi-user trading desk with shared accounts, strategies, and risk policy.
- A coaching organization with isolated client workspaces and delegated review permissions.

- An enterprise trading or research organization with SSO, team hierarchies, environment separation, and policy inheritance.
- A Hermes operator tenant managing providers, chains, pools, and incidents.
- A hybrid tenant using both trading and Hermes capabilities.
- A platform internal tenant for support and system operations, separated from customer data by explicit privileged workflows.

A user may belong to multiple tenants. Membership, role, permissions, team assignment, environment access, and account scope are tenant-specific. No user-level cache or client state may assume a single tenant.

Role and Permission Model

RBAC supplies stable role bundles; ABAC supplies resource and context conditions. The permission decision combines:

- Tenant membership and status.
- Role and explicit permission grants.
- Team, account, portfolio, strategy, provider, chain, and environment scope.
- Time, network, device, authentication strength, and approval state.
- Policy version and requested execution mode.
- Resource classification and action sensitivity.

Representative roles include viewer, trader, senior_trader, coach, risk_manager, portfolio_manager, liquidity_analyst, operator, compliance_reviewer, tenant_admin, and platform_admin. Custom roles shall be supported without schema changes.

Operating Environments

Every tenant may have logical environments such as sandbox, paper, staging, and production. Credentials, providers, policies, data retention, and execution permissions are environment-scoped. The same user may have different rights in each environment. Cross-environment data copying requires an audited export and import process; production secrets and live account identifiers are not automatically propagated.

Collaboration Model

The product shall support comments, mentions, assignments, review queues, approvals, evidence attachments, and immutable decision records. Collaboration objects are linked to canonical resources rather than stored only in chat transcripts. Notifications are delivered through in-app inbox, email, push, webhook, and configured messaging adapters.

Product Scope and Capability Catalog

Track A - AI Trading Operating System

Track A comprises the following first-class modules. Each module is available through web interfaces and typed APIs, and each produces auditable domain artifacts.

Capability	Required behavior	Principal outputs
Trade ingestion and journal	Import manual records, CSV, broker history, exchange history, executions, fees, screenshots, and notes; normalize and reconcile	Canonical trades, fills, positions, tags, source lineage, reconciliation status
Journal analyzer	Evaluate setup, timing, adherence, context, outcome, and behavior	Analysis report, rule matches, strengths, errors, improvement actions
Risk manager	Size positions, enforce limits, monitor exposure, and manage breaches	Risk decision, approved size, warnings, blocks, approvals, breach records

Capability	Required behavior	Principal outputs
Market briefing	Produce scheduled and on-demand market context	Briefing, scenarios, event risks, watchlists, evidence links
AI trading coach	Detect behavioral patterns and manage improvement plans	Coaching observations, goals, exercises, review schedule, progress
Backtesting	Run reproducible historical strategy tests	Run manifest, trades, metrics, assumptions, artifacts, comparison
Regime detection	Classify market conditions across assets and horizons	Regime labels, probabilities, transitions, supporting features
Screenshot recognition	Parse charts, orders, annotations, and interface captures	Extracted symbols, levels, indicators, text, chart context, confidence
Trade replay	Reconstruct market and decision sequence around a trade	Timeline, available information, alternate actions, annotations
Probability engine	Estimate calibrated scenario or setup likelihoods	Probability distribution, calibration metadata, features, uncertainty
Playbook builder	Create versioned strategy and execution rules	Playbook versions, setup criteria, invalidation, risk rules, checklists
Trade grading	Score process and outcome separately	Grade dimensions, weighted score, evidence, peer or policy benchmark
Emotional tracking	Capture and correlate mental state with behavior and outcomes	State records, triggers, trends, intervention prompts
Risk scaling	Adjust risk allocation using performance, volatility, drawdown, and policy	Scale factor, rationale, effective window, approval status
Smart watchlist	Rank instruments and setups from explicit criteria	Ranked candidates, reason codes, alerts, freshness, confidence
Strategy discovery	Mine records and market data for repeatable patterns	Candidate hypotheses, supporting sample, robustness tests, caveats
Portfolio analytics	Aggregate risk, return, exposure, correlation, and scenario impact	Portfolio state, attribution, stress results, concentration alerts
Order and execution management	Create, route, amend, cancel, reconcile, and analyze orders	Order states, fills, rejects, slippage, venue data, execution evidence
Research workspace	Organize hypotheses, notes, datasets, reports, and experiments	Research projects, notebooks, references, result artifacts
Reporting and export	Produce scheduled, interactive, and regulatory-ready reports	PDF, CSV, JSON, evidence package, signed export manifest

Track B - Hermes Protocol

Hermes is the multi-chain liquidity intelligence and operations subsystem. It uses the same tenant, policy, event, audit, AI, observability, and deployment foundation as Track A.

Capability	Required behavior	Principal outputs
Chain and network registry	Define chains, networks, finality, native assets, explorers, and RPC requirements	Versioned network configuration
Provider registry	Manage RPC, indexer, oracle, market-data, and quote providers	Provider profiles, endpoints, credentials references, capabilities
Provider health	Poll and score availability, latency, freshness, error rate, completeness, and divergence	Health samples, scores, SLO status, incidents

Capability	Required behavior	Principal outputs
Quorum engine	Compare multiple provider observations and produce a confidence-rated canonical result	Quorum decision, included and excluded observations, evidence
Token and pool registry	Normalize tokens, contracts, decimals, pools, venues, and pair relationships	Canonical assets and liquidity venues
Liquidity snapshots	Capture reserve, depth, spread, price impact, volume, and fee information	Time-series liquidity observations
Route intelligence	Evaluate candidate routes across venues and chains	Ranked routes, expected output, impact, gas, confidence, failure modes
Anomaly detection	Detect liquidity loss, price divergence, stale feeds, provider faults, and unusual activity	Alert, severity, evidence, affected resources
Operator dashboard	Provide health, quorum, incident, provider, chain, pool, and route views	Operational command center
Incident management	Open, assign, investigate, mitigate, resolve, and review incidents	Incident timeline, actions, communications, postmortem
Evidence and compliance export	Package configuration, tests, health history, decisions, and approvals	Signed evidence bundle and manifest
Automation and runbooks	Execute policy-authorized remediation and diagnostic workflows	Runbook runs, tool calls, outcomes, rollback evidence

Shared Platform Capabilities

Shared services include identity, tenancy, RBAC and ABAC, audit, policy, notifications, files, search, scheduling, event delivery, integration management, secrets references, billing hooks, usage metering, feature configuration, data export, data deletion, support workflows, and developer tooling. These services are not duplicated between tracks.

Capability Exposure

The application shall use capability discovery rather than static navigation assumptions. A tenant session receives a signed capability manifest based on installed modules, policy, permissions, environment, and integration state. The manifest controls visible routes and actions, but the backend remains authoritative. Hiding a UI control is never treated as authorization.

End-to-End Workflows and Functional Requirements

Trader Onboarding and Data Activation

1. A user creates or joins a tenant and selects an operating environment.
2. The user configures profile, timezone, base currency, preferred markets, and notification channels.
3. The user connects one or more brokers or exchanges, uploads files, or begins with manual records.
4. The integration service discovers accounts and instrument metadata without persisting raw credentials in application tables.
5. Imports create immutable source batches and normalized candidate records.
6. Reconciliation identifies duplicates, partial fills, missing fees, symbol mappings, and time-zone ambiguities.
7. The user approves or corrects unresolved items.
8. The system calculates opening analytics and builds an initial coaching and risk baseline.

Acceptance criteria include deterministic duplicate detection, source lineage on every imported fact, tenant-isolated storage, resumable imports, and a complete reconciliation report.

Daily Trading Cycle

The daily cycle connects briefing, watchlist, risk, execution, and review:

- Before the session, the briefing agent assembles scheduled events, overnight movement, market structure, regimes, volatility, liquidity, and tenant playbooks.
- The watchlist service ranks candidates using explicit filters and model-assisted context.
- A trader creates an intent containing instrument, direction, thesis, setup, entry plan, invalidation, targets, expected duration, and requested risk.
- The risk engine evaluates the intent against account, portfolio, strategy, instrument, market, and behavioral policies.
- An authorized user or agent creates an order plan. The order service validates market data freshness, buying power, venue constraints, idempotency, and risk decision validity.
- The execution adapter submits the order and streams acknowledgements, partial fills, fills, rejects, cancels, and corrections.
- The position service updates derived state; reconciliation compares internal and external records.
- During and after the trade, the journal captures notes, screenshots, emotions, rule deviations, and market context.
- The analyzer and coach produce evidence-linked feedback after the trade closes or at a configured checkpoint.

Risk Breach and Approval Workflow

A risk check returns one of `allow`, `allow_with_warning`, `require_approval`, or `block`. A warning requires explicit acknowledgement. An approval creates a time-limited request containing the proposed action, computed exposure, violated or near-limit rules, rationale, and approver scope. A block cannot proceed unless the applicable policy itself permits an override and the actor has override authority. All acknowledgements, overrides, and approvals are append-only audit events.

Weekly Review and Coaching Workflow

The system prepares a weekly evidence set containing performance, process grades, rule adherence, emotional patterns, regime segmentation, execution quality, missed opportunities, and open coaching goals. The coach agent proposes a limited set of high-impact actions. The user or coach accepts, edits, or rejects each action. Accepted actions become measurable plan items with due dates, triggers, and follow-up evidence.

Hermes Provider Incident Workflow

1. Health collectors detect an SLO breach, divergence, stale response, malformed payload, or availability failure.
2. The provider-quality service opens or updates an incident using deduplication keys.
3. The quorum engine removes or down-weights affected observations according to policy.
4. Operators receive a prioritized incident with affected chains, endpoints, pools, routes, tenants, and decisions.
5. A runbook may perform diagnostics, rotate endpoints, change weights, or fail over when authorized.
6. The system records recovery, validates data consistency, and produces a post-incident report.

Core Functional Requirement Families

Requirement family	Mandatory requirements
FR-ID	Multi-tenant identity, membership, MFA, SSO-ready authentication, session management, service identities, and scoped API keys
FR-DATA	Canonical records, source lineage, versioning, reconciliation, retention, export, deletion, and time-zone correctness
FR-JRN	Trade import, manual entry, notes, screenshots, tags, replay, grading, and review queues

Requirement family	Mandatory requirements
FR-RISK	Policy authoring, pre-trade and continuous checks, approvals, overrides, alerts, stress, and audit
FR-MKT	Market data normalization, freshness, calendars, news, regimes, watchlists, and briefings
FR-EXEC	Account discovery, order lifecycle, routing, amendments, cancels, fills, reconciliation, and multiple execution modes
FR-AI	Agent registry, tool permissions, structured outputs, model routing, retrieval, memory, evaluation, and provenance
FR-HERMES	Provider registry, health, quorum, chain data, liquidity snapshots, routes, anomalies, incidents, and evidence
FR-OPS	Scheduling, jobs, notifications, support, usage metering, audit search, environment management, and exports
FR-SEC	RLS, RBAC and ABAC, secrets, encryption, security events, vulnerability management, and incident response

Universal Acceptance Rules

Every user-visible workflow shall meet the following criteria:

- The workflow is available through a documented API or domain command, not only a UI.
- Errors are actionable and include a stable code, correlation identifier, and retry guidance.
- Long-running work is resumable and exposes progress, status, and failure evidence.
- All displayed market, model, and provider outputs show freshness and provenance.
- Sensitive changes are audited with actor, tenant, resource, before and after references, and reason.
- Accessibility, localization readiness, keyboard operation, and responsive behavior are included in completion.
- Telemetry, dashboards, alerts, runbooks, and tests are delivered with the feature.

Unified System Architecture

Architectural Style

The platform uses a modular, event-driven architecture with explicit domain boundaries. The web layer presents trader, operator, administrator, and developer experiences. Synchronous APIs validate commands and serve transactional queries. Workers perform imports, reconciliation, provider polling, analytics, AI runs, report generation, backtests, and scheduled workflows. A transactional outbox connects database commits to asynchronous domain events so that state and evidence cannot diverge.

The initial deployment may be a modular monolith for API and domain services, provided package boundaries and data ownership are enforced. Services may be extracted when load, availability, deployment cadence, or organizational ownership justifies separation. Extraction shall not change public contracts or weaken tenant enforcement.

Logical Layer Model

Layer	Responsibilities	Representative components
Experience	Navigation, visualization, forms, real-time status, accessibility	Trader dashboard, journal, risk console, coach, Hermes operations, admin portal
Application	Command orchestration, queries, authorization, workflow coordination	REST API, GraphQL query facade, BFF modules, realtime gateway
Domain	Business rules, state machines, calculations, policy evaluation	Trading, journal, risk, portfolio, market, AI, Hermes, incident domains
Agents	Model orchestration, retrieval, tools, structured outputs, evaluations	Analyst, briefing, coach, risk, research, liquidity, incident agents
Data	Transactional data, files, vectors, cache, event transport, analytics	Supabase Postgres, storage, vector indexes, Redis-compatible cache, outbox, warehouse
Integration	Broker, exchange, market-data, model, notification, and chain adapters	Adapter SDK, connector runtime, webhook gateway, secrets references
Engineering control	Specification, repository policy, approval, traceability, command execution, verification	Spec Kit artifacts, AGENTS.md, codex-hitl, SQLite local audit, CI evidence
Operations	Deployment, telemetry, security, support, evidence, recovery	CI/CD, infrastructure as code, logs, metrics, traces, backups, runbooks

Runtime Components

Web applications

- apps/web: primary Next.js application for traders, coaches, risk staff, and tenant administrators.
- apps/hermes-ops: optional independently deployable operator shell for Hermes workflows.
- apps/platform-admin: privileged platform operations and support console.
- apps/docs: developer portal, API reference, integration guides, and system status.

These applications may share components and route groups, but privileged surfaces shall be separately authorized and deployable.

API and realtime services

- apps/api: typed command and query endpoints, authorization, idempotency, validation, and transaction coordination.
- apps/realtime: WebSocket or server-sent-event delivery for order state, provider health, jobs, incidents, and notifications.

- `apps/webhooks`: inbound verification and outbound delivery with signing, retries, and dead-letter handling.

Worker and scheduler services

- `apps/workers`: asynchronous domain and integration jobs.
- `apps/quant-workers`: Python services for computationally intensive analytics, optimization, simulation, and model pipelines.
- `apps/scheduler`: recurring jobs, market-calendar-aware schedules, dependency handling, and backfills.
- `apps/connector-runtime`: isolated execution for third-party adapters with bounded permissions and resource limits.

Engineering control tooling

- `tooling/codex-hitl`: Python command-line control plane for deterministic repository policy, approvals, workflow state, bounded command execution, SQLite history, traceability, and verification.
- `.specify` and `specs`: constitution, feature specifications, plans, tasks, contracts, and acceptance checklists.
- `AGENTS.md`: repository and directory-specific Codex operating rules.
- `policies`: versioned TOML policies and schemas for engineering actions.

Domain Boundaries

The principal domains are identity, tenancy, policy, integrations, market data, trading, portfolio, risk, journal, analytics, AI, Hermes, incidents, notifications, files, audit, and reporting. Each domain owns its write model and emits events for cross-domain reactions. Direct cross-domain table mutation is prohibited. Read models may join across domains in controlled query modules or materialized views.

Examples:

- Trading owns orders, executions, and account snapshots.
- Portfolio owns positions, allocations, and aggregate exposures derived from trading events.
- Journal owns user-authored context and review artifacts linked to trades.
- Risk owns policy, evaluations, breaches, approvals, and overrides.
- AI owns agent definitions, runs, outputs, feedback, and evaluation records, but not the canonical resources it analyzes.
- Hermes owns provider observations, quorum decisions, liquidity records, and incidents specific to liquidity infrastructure.

Reference Request and Event Flow

1. A client sends a command with tenant context, idempotency key, expected resource version, and correlation identifier.
2. The API authenticates the principal, resolves membership and environment, evaluates authorization, and validates the request schema.
3. The application service loads domain state and applicable policy versions.
4. The domain handler evaluates invariants and returns state changes plus domain events.
5. A single database transaction writes the state, audit record, and outbox events.
6. The API returns the committed resource version and command result.
7. The outbox publisher delivers events to the event transport.
8. Consumers update projections, trigger notifications, run agents, or call integrations.
9. Realtime channels notify subscribed clients with sequence and replay information.
10. Observability links the HTTP request, transaction, event, worker job, integration call, and agent run under one trace.

Deployment Topology

A reference production topology includes:

- A CDN and web application edge for static assets and authenticated server rendering.
- A regional API cluster behind a managed load balancer and web application firewall.
- Separate worker pools for general jobs, latency-sensitive execution, AI workloads, and Hermes collectors.
- Supabase Postgres configured for point-in-time recovery, read replicas where required, connection pooling, and row-level security.
- Object storage for screenshots, statements, backtest artifacts, reports, and evidence bundles.
- A Redis-compatible cache for ephemeral sessions, rate counters, distributed locks, and short-lived market data.
- A durable queue or event bus for asynchronous delivery, retries, and dead letters.
- Central logs, metrics, traces, error tracking, and security-event ingestion.
- A secrets manager or vault used by integration runtimes through short-lived credentials.

The architecture remains portable. Vendor-specific services are wrapped behind deployment modules and interfaces.

Monorepo Structure

AGENTS.md	repository-wide Codex operating contract
.specify/	constitution and Spec Kit project memory
specs/	feature specs, plans, tasks, contracts, checklists
policies/	versioned engineering TOML policies and schemas
apps/	
web/	trader, coach, risk, and tenant administration UI
hermes-ops/	liquidity operations UI
platform-admin/	privileged platform console
api/	commands, queries, authorization, transactions
realtime/	WebSocket or SSE gateway
webhooks/	inbound and outbound webhook service
workers/	general asynchronous jobs
quant-workers/	Python analytics and simulation services
scheduler/	recurring and market-calendar workflows
connector-runtime/	isolated third-party adapter execution
docs/	developer and operational documentation
packages/	
core/	identifiers, money, time, errors, shared schemas
identity/	tenants, membership, roles, permissions
policy/	policy model, DSL, evaluator, approvals
trading/	accounts, orders, executions, reconciliation
portfolio/	positions, allocation, exposure, attribution
risk/	sizing, limits, breaches, stress calculations
journal/	trades, notes, screenshots, replay, grading
analytics/	metrics, regimes, probability, backtests
market-data/	symbols, quotes, candles, calendars, news
ai/	agents, tools, prompts, model router, evaluations
hermes/	chains, providers, quorum, pools, routes, incidents
integrations/	broker, exchange, data, model, chain adapters
events/	event envelope, outbox, consumers, schemas
observability/	logging, metrics, tracing, audit helpers
ui/	design system and shared components
testing/	fixtures, factories, contract and RLS helpers
config/	environment and tenant configuration schemas
tooling/	
codex-hitl/	policy, approval, workflow, executor, audit, verify
supabase/	
migrations/	forward-only schema changes
seed/	local and test data
tests/	RLS, functions, and migration verification
infra/	
modules/	reusable infrastructure components
environments/	dev, preview, staging, production definitions
policies/	deployment and security policy as code
docs/	
adr/	architecture decision records
runbooks/	operational procedures
threat-models/	system and feature threat models

evidence/ .github/workflows/ Dockerfile SECURITY.md	generated readiness artifacts CI, security, evidence, and deployment workflows reproducible starter and verification image vulnerability and security operating policy
--	---

Dependency Rules

- packages/core has no dependency on application or integration packages.
- Domain packages depend only on core abstractions and explicitly shared value objects.
- Integration packages implement domain ports; domain packages never import vendor SDKs.
- UI packages consume public API contracts and view models, not database types.
- AI packages call domain tools through typed interfaces and do not write canonical tables directly.
- Cross-package cycles fail CI.
- Database access is isolated in repositories with tenant-aware query construction and test fixtures.
- Repository automation depends on tooling/codex-hitl through its CLI or typed interfaces; no package may bypass policy, approval, workflow, or audit persistence for governed commands.
- Specifications, tasks, source, tests, and evidence must preserve valid requirement links; traceability failures block completion.

Architecture Decision Records

Material choices shall be recorded as ADRs, including context, options, decision, consequences, rollback, and review date. Required ADR topics include API style, event transport, cache, market-data storage, vector search, secrets, model providers, broker adapter isolation, chain finality handling, analytics warehouse strategy, Codex sandbox profile, engineering policy semantics, approval-token verification, audit migration, and command-execution isolation.

Domain Model and Data Architecture

Data Principles

The data architecture uses Postgres as the transactional source of truth. Derived analytics, search indexes, vector embeddings, caches, and warehouse tables can be rebuilt from canonical records and events. Every record uses stable opaque identifiers, UTC timestamps, explicit source and effective-time fields where applicable, and tenant ownership unless declared global reference data.

Financial values are stored using decimal or integer minor-unit representations, never binary floating point in transactional calculations. Instrument quantities and prices include scale and currency or asset identifiers. Time-series facts distinguish event time, provider time, ingestion time, and processing time.

Identity and Tenancy Entities

Entity	Purpose	Key fields
tenants	Customer or internal workspace	id, name, type, status, default_timezone, base_currency
users	Platform user profile linked to auth identity	id, auth_subject, locale, status
tenant_members	User membership in a tenant	tenant_id, user_id, status, joined_at
roles	Tenant or system role definition	id, tenant_id nullable, name, version
permissions	Stable action identifiers	code, resource_type, action
role_permissions	Role bundle membership	role_id, permission_id, conditions
member_roles	Role assignment with scope	member_id, role_id, environment_id, scope_json
service_identities	Machine principals	id, tenant_id, name, key_reference, status

Entity	Purpose	Key fields
api_keys	Hashed, scoped API credentials	id, principal_id, prefix, hash, expires_at, scopes
environments	Sandbox, paper, staging, or production context	id, tenant_id, code, execution_mode, status

Trading and Portfolio Entities

Entity	Purpose
venues and instruments	Canonical exchanges, brokers, chains, symbols, contracts, and instrument metadata
integration_connections	Tenant connector configuration and secret references
trading_accounts	Broker or exchange account identity, currency, capabilities, and environment
orders	Canonical order intent and lifecycle state
order_legs	Multi-leg order components
executions	External acknowledgements, fills, corrections, and fees
positions	Current account and portfolio position projections
position_lots	Tax or accounting lots where enabled
portfolios	Logical aggregation of accounts, strategies, and capital
allocations	Capital or exposure assignment to strategies, teams, or accounts
strategies	Versioned strategy definitions and ownership
playbooks and playbook_versions	Setup, entry, exit, invalidation, risk, and checklist rules
trade_intents	Pre-trade thesis and plan before order creation
trades	Analytical trade grouping across orders and fills
reconciliation_runs and reconciliation_items	Internal versus external consistency evidence

Journal, Coaching, and Research Entities

The journal domain includes `journal_entries`, `trade_notes`, `tags`, `trade_tags`, `attachments`, `screenshots`, `chart_extractions`, `emotional_states`, `rule_observations`, `trade_grades`, `replay_sessions`, `replay_events`, `coach_observations`, `improvement_plans`, `plan_items`, `reviews`, `comments`, and `assignments`.

Research uses `research_projects`, `hypotheses`, `datasets`, `experiments`, `backtest_runs`, `backtest_parameters`, `backtest_trades`, `result_artifacts`, and `comparisons`. Every generated result records code version, data snapshot or query, parameter set, random seed where applicable, and runtime environment.

Risk and Policy Entities

Entity	Purpose
policy_sets	Named collection of policy rules for a tenant and environment
policy_versions	Immutable policy publication with effective dates
policy_rules	Conditions, thresholds, actions, and reason codes
risk_evaluations	Reproducible evaluation for an intent, order, position, or portfolio
risk_results	Per-rule outcome, value, limit, severity, and evidence
risk_breaches	Open or historical limit breach
approval_requests	Time-limited approval for a proposed action

Entity	Purpose
approval_actions	Approve, deny, request change, revoke, or expire
overrides	Explicit policy-authorized exception with rationale and scope
kill_switches	Tenant, environment, account, strategy, instrument, or provider stop state

Market and Analytics Entities

Market reference data includes calendars, sessions, instruments, corporate actions, economic events, and news references. Time-series tables include quotes, trades, bars, order-book snapshots, volatility surfaces, funding rates, open interest, and derived features as supported by providers.

Analytics entities include metric_definitions, metric_values, regime_models, regime_observations, probability_models, probability_predictions, calibration_runs, watchlists, watchlist_candidates, alerts, scenario_sets, and stress_results.

High-volume time-series tables shall be partitioned by time and, where useful, instrument or provider. Retention policies can downsample or archive data while preserving reproducibility manifests.

AI Entities

Entity	Purpose
models	Logical and vendor-specific model catalog
model_endpoints	Provider endpoint, region, capability, and cost configuration
agents and agent_versions	Agent purpose, instructions, tools, schemas, and policies
prompt_templates and prompt_versions	Versioned system, task, and formatting prompts
agent_runs	Complete invocation record with tenant, actor, model, inputs, and status
tool_calls	Authorized tool request, arguments, outcome, and timing
agent_outputs	Validated structured result and rendered presentation
agent_feedback	User or evaluator rating, correction, and disposition
evaluation_suites and evaluation_results	Offline and online quality evidence
knowledge_documents, chunks, and embeddings	Tenant-scoped retrieval corpus and vector index
memory_items	Explicit, scoped, expiring agent memory records

Hermes Entities

Hermes uses chains, networks, rpc_providers, provider_endpoints, provider_capabilities, provider_health_samples, provider_scores, provider_incidents, tokens, token_contracts, liquidity_venues, pools, pool_snapshots, oracle_observations, quote_requests, quote_observations, quorum_decisions, routes, route_legs, route_evaluations, liquidity_alerts, and operator_runbook_runs.

Every on-chain observation includes network, block number or slot, block hash where available, finality status, provider, request parameters, response hash, event time, and ingestion time. Reorganizations create correction events and invalidate affected derived states rather than overwriting history silently.

Operations and Evidence Entities

Operational tables include jobs, job_attempts, outbox_events, inbox_deduplication, webhook_deliveries, notifications, audit_events, security_events, deployment_records, feature_policies, configuration_versions, evidence_packages, and export_manifests.

Audit events are append-only and contain:

```
{
  "id": "aud_...",
  "tenant_id": "ten_...",
  "environment_id": "env_...",
  "occurred_at": "2026-07-10T14:30:00Z",
  "actor": {"type": "user", "id": "usr_...", "session_id": "ses_..."},
  "action": "order.submit",
  "resource": {"type": "order", "id": "ord_...", "version": 4},
  "decision": {"authorization": "allow", "policy_version": "polv_..."},
  "correlation_id": "cor_...",
  "request_hash": "sha256:...",
  "before_ref": "snapshot:...",
  "after_ref": "snapshot:...",
  "reason": "Approved strategy signal",
  "metadata": {}
}
```

Row-Level Security Pattern

All tenant-owned tables include `tenant_id` and use RLS policies based on a database session claim or verified request context. The application shall not rely only on query filters.

Representative policy pattern:

```
alter table trades enable row level security;

create policy trades_select_member
on trades for select
using (
  tenant_id = app.current_tenant_id()
  and app.has_permission('trade.read', id)
);

create policy trades_write_member
on trades for all
using (
  tenant_id = app.current_tenant_id()
  and app.has_permission('trade.write', id)
)
with check (
  tenant_id = app.current_tenant_id()
  and app.has_permission('trade.write', id)
);
```

Service-role access is limited to narrowly scoped worker functions, and every bypass path is tested and audited.

Versioning and Concurrency

Mutable aggregates include an integer version used for optimistic concurrency. Commands provide the expected version. Conflicts return a stable error containing the current version and safe retry instructions. Policy, prompt, playbook, agent, connector, and model configurations are immutable after publication; changes create a new version with effective dates.

Retention, Export, and Deletion

Retention is configured by data class, tenant, environment, legal hold, and contract. Deletion workflows distinguish user-visible deletion, recoverable retention, cryptographic erasure of secrets, and immutable evidence that must be retained under applicable policy. Exports include a manifest, schema version, checksums, source lineage, timezone, currency conventions, and generation audit event.

API, Event, and Integration Contracts

API Strategy

The platform uses versioned REST commands and resource endpoints as the authoritative public API. A GraphQL query facade may provide flexible read access for internal and partner clients. Realtime delivery uses WebSocket or server-sent events. Asynchronous external integration uses signed webhooks and event subscriptions.

All APIs are generated from typed schemas and published through OpenAPI, JSON Schema, and SDK packages. Internal and external consumers use the same domain identifiers and error catalog.

Request Conventions

Every mutating request includes:

- Authorization or a signed service credential.
- X-Tenant-Id and X-Environment-Id where not embedded in the path.
- Idempotency-Key for commands that can be retried.
- X-Correlation-Id or a server-generated identifier.
- Optional If-Match or expected version for concurrency.
- A validated JSON body with explicit units, currencies, timezones, and instrument identifiers.

Responses include resource version, correlation identifier, audit reference when applicable, and warnings. Pagination uses opaque cursors. Dates are ISO 8601 UTC unless an endpoint explicitly represents a local trading session.

Error Contract

```
{
  "error": {
    "code": "RISK_APPROVAL_REQUIRED",
    "message": "The proposed order requires risk approval.",
    "correlation_id": "cor...",
    "retryable": false,
    "details": {
      "approval_request_id": "apr...",
      "risk_evaluation_id": "rsk...",
      "reason_codes": ["DAILY_LOSS_NEAR_LIMIT", "POSITION_CONCENTRATION"]
    }
  }
}
```

Errors are stable across UI and SDKs. Vendor errors are mapped to platform codes while preserving sanitized provider details for operators.

Representative Endpoint Catalog

Area	Representative endpoints
Identity	GET /v1/session, GET /v1/tenants, POST /v1/members, POST /v1/roles
Integrations	POST /v1/connections, POST /v1/connections/{id}/discover, POST /v1/imports
Journal	GET /v1/trades, POST /v1/trades, POST /v1/trades/{id}/analyze, POST /v1/replays
Risk	POST /v1/risk/evaluations, POST /v1/approvals, POST /v1/approvals/{id}/actions, POST /v1/kill-switches

Area	Representative endpoints
Market	GET /v1/market/quotes, GET /v1/market/bars, POST /v1/briefings, POST /v1/watchlists/{id}/refresh
Execution	POST /v1/order-intents, POST /v1/orders, POST /v1/orders/{id}/cancel, POST /v1/orders/{id}/replace
AI	POST /v1/agent-runs, GET /v1/agent-runs/{id}, POST /v1/agent-runs/{id}/feedback
Analytics	POST /v1/backtests, POST /v1/scenarios, GET /v1/metrics, POST /v1/probabilities
Hermes	GET /v1/hermes/providers, POST /v1/hermes/quorum, GET /v1/hermes/pools, POST /v1/hermes/routes
Incidents	POST /v1/incidents, POST /v1/incidents/{id}/actions, POST /v1/runbooks/{id}/execute
Evidence	POST /v1/evidence-packages, GET /v1/audit-events, POST /v1/exports

Command Idempotency

The server stores idempotency keys with tenant, environment, principal, command type, canonical request hash, result reference, and expiry. Reuse with the same request returns the prior result. Reuse with a different request returns IDEMPOTENCY_CONFLICT. External order identifiers and provider request identifiers are included in deduplication rules.

Event Envelope

```
{
  "event_id": "evt...",
  "event_type": "order.filled.v1",
  "occurred_at": "2026-07-10T14:31:22.441Z",
  "published_at": "2026-07-10T14:31:22.512Z",
  "tenant_id": "ten...",
  "environment_id": "env...",
  "aggregate": {"type": "order", "id": "ord...", "version": 7},
  "actor": {"type": "service", "id": "svc_execution"},
  "correlation_id": "cor...",
  "causation_id": "evt...",
  "schema_version": 1,
  "data": {},
  "provenance": {"source": "broker_adapter", "connection_id": "con..."}
}
```

Consumers are at-least-once and must be idempotent. Event schemas are backward compatible within a major version. Breaking changes require a new event type version and a migration plan.

Event Families

Events include identity and membership changes, configuration publication, import progress, trade lifecycle, order lifecycle, position changes, risk decisions, breaches, approvals, briefing generation, agent runs, model evaluations, provider health, quorum decisions, chain observations, liquidity alerts, incidents, deployments, exports, and security events.

Webhooks

Outbound webhooks are signed with rotating keys, include timestamp and delivery identifiers, and support replay within a bounded window. Delivery uses exponential backoff, configurable retry policy, tenant-specific endpoints, event filters, and a dead-letter queue. The console exposes attempts, response codes, latency, and manual replay.

Inbound webhooks verify signature, timestamp, source IP policy where available, event identity, and schema. Raw payloads are stored only when policy allows and are encrypted with restricted access.

Adapter Contracts

Every adapter declares capabilities, supported markets or networks, authentication method, rate limits, order types, data types, pagination behavior, webhook support, sandbox availability, and known limitations. Adapters implement health checks and contract tests against recorded fixtures and provider sandboxes.

The domain layer communicates through ports such as:

```
export interface ExecutionVenuePort {
  discoverAccounts(input: DiscoverAccountsInput): Promise<AccountDescriptor[]>;
  submitOrder(command: SubmitVenueOrder): Promise<VenueOrderAcknowledgement>;
  replaceOrder(command: ReplaceVenueOrder): Promise<VenueOrderAcknowledgement>;
  cancelOrder(command: CancelVenueOrder): Promise<VenueOrderAcknowledgement>;
  getOrderSnapshot(query: VenueOrderQuery): Promise<VenueOrderSnapshot>;
  listExecutions(query: VenueExecutionQuery): Promise<VenueExecutionPage>;
  getCapabilities(): Promise<VenueCapabilities>;
  healthCheck(): Promise<AdapterHealth>;
}
```

SDK and Developer Experience

Official TypeScript and Python SDKs shall provide authentication, retries, pagination, idempotency, event verification, and typed resources. A sandbox includes deterministic fixtures, simulated market data, paper accounts, mock chain providers, and webhook inspection. Integration certification produces a machine-readable capability and test report.

Trading Connectivity, Order Management, and Execution

Execution Modes

The build supports a unified progression of execution modes:

Mode	Behavior
Observe	Import accounts, orders, executions, positions, and balances without creating orders
Simulate	Execute against a deterministic or stochastic simulator using historical or synthetic market data
Paper	Submit to a venue-provided or platform-provided paper environment
Live manual	User explicitly confirms each live order submission
Live supervised	Agents or strategies prepare orders; an authorized user approves according to policy
Live automated	Policy-authorized strategies or agents submit and manage orders without per-order confirmation

The same order model and risk path apply to every mode. The environment and account configuration determine the selected adapter and permission requirements.

Canonical Order Model

An order includes tenant, environment, account, portfolio, strategy, instrument, side, quantity, quantity type, order type, limit or stop price, time in force, session, execution instructions, parent and child relationships, requested risk, source, automation context, and client idempotency key. Multi-leg orders use an aggregate order with leg ratios and execution constraints.

Order state is modeled as a state machine, not free-form status text. Representative states are draft, risk_pending, approval_pending, approved, submitting, acknowledged, partially_filled, filled, cancel_pending, canceled, replace_pending, rejected, expired, suspended, and reconciliation_required.

Pre-Trade Processing

Before submission, the order service shall:

1. Resolve instrument, venue, session, account capability, and current market state.
2. Verify quote or reference-price freshness and provider confidence.
3. Calculate notional, estimated fees, margin, buying power, slippage, and worst-case exposure.
4. Evaluate applicable risk policies and kill switches.
5. Confirm approval and execution-mode requirements.
6. Validate order type, tick size, lot size, price bands, and venue constraints.
7. Reserve internal risk capacity where policy requires.
8. Persist the order, audit event, and outbox command transactionally.
9. Submit through the adapter using stable client and provider identifiers.

Risk evaluations expire when market, account, policy, or order inputs change beyond configured tolerances.

Execution Gateway

The execution gateway is isolated from general AI and web workloads. It uses short-lived credential access, bounded network egress, strict schemas, low-latency queues, and dedicated observability. No agent has direct access to broker SDKs; agents invoke an authorized order command through the same API used by human clients.

The gateway shall support:

- Venue capability discovery and normalized order translation.
- Client-order-id generation and duplicate prevention.
- Retry rules that distinguish safe lookup retries from unsafe submission retries.
- Circuit breakers for provider failures or error-rate spikes.
- Streaming and polling reconciliation.
- Clock synchronization and latency measurement.
- Kill switches at global, tenant, environment, account, strategy, instrument, and adapter scope.
- A degraded mode that stops new exposure while allowing cancels and reconciliation.

Smart Routing and Allocation

Where multiple venues or accounts are eligible, a routing service may evaluate expected fill, price, liquidity, fees, latency, venue health, account capacity, policy, and concentration. Routing decisions are versioned and auditable. The system supports single venue, best-price, split allocation, sequential fallback, and custom policy strategies.

Routing never assumes that a quoted price is executable. It carries freshness, size, depth, and expected slippage. For on-chain routes, it also includes gas, block freshness, finality, pool state, price impact, token transfer behavior, and transaction simulation results.

Fill and Position Processing

Venue events are normalized into acknowledgements, fills, fee records, corrections, busts, and rejects. Position updates are derived from execution events and reconciled against venue snapshots. The system handles partial fills, out-of-order events, duplicate webhooks, currency conversion, corporate actions, options exercises and assignments, futures rolls, funding, and on-chain settlement states through instrument-specific extensions.

Reconciliation

Reconciliation runs on schedule and after material exceptions. It compares internal orders, executions, positions, cash, balances, fees, and status against the external system. Differences are classified, assigned severity, and linked to remediation workflows. Live automated execution is suspended for affected scopes when policy thresholds are exceeded.

Automation Controls

Automation is governed by policy, not removed from the build. Policies can restrict or allow:

- Strategies, agents, accounts, instruments, venues, sessions, and order types.
- Maximum per-order, daily, portfolio, leverage, and concentration exposure.
- Required data providers and minimum confidence.
- Approval thresholds and approver separation.
- Trading windows, cooldowns, loss limits, and anomaly responses.
- Order amendment, cancel, liquidation, and emergency behavior.

Each automated command records the strategy or agent version, signal or analysis artifact, tool authorization, risk evaluation, policy version, model version where applicable, and resulting venue evidence.

Execution Quality Analytics

The platform calculates arrival price, decision price, implementation shortfall, spread capture, slippage, market impact, fill rate, time to fill, cancel rate, reject rate, fee burden, adverse selection, and venue comparison. Metrics are segmented by strategy, instrument, order type, time, volatility, liquidity, provider state, and automation mode.

Failure and Recovery Cases

Explicit recovery paths are required for timeouts after submission, duplicate acknowledgements, missing webhooks, partial venue outages, stale account state, adapter version defects, provider maintenance, clock drift, network partitions, and process restarts. The system shall prefer reconciliation over blind retries and shall surface uncertain state as `reconciliation_required` until resolved.

Risk Management and Quantitative Analytics

Risk Architecture

Risk is a dedicated domain service with deterministic calculations, versioned policy, and independent audit. It evaluates proposed trades, orders, positions, portfolios, strategies, accounts, and tenant-wide exposure. It also consumes continuous market and account events to detect breaches after execution.

The engine separates three concerns:

1. **Measurement** computes exposures and metrics from canonical state.
2. **Policy** compares measurements and context against versioned rules.
3. **Decision** combines rule outcomes into allow, warn, approve, block, reduce, cancel, or liquidate actions.

Policy Hierarchy

Policies can exist at platform, tenant, environment, team, portfolio, account, strategy, instrument group, instrument, and user scope. More specific policies may tighten or, when explicitly permitted, relax inherited values. Effective policy resolution returns the complete rule set and provenance for every value.

Rules support hard thresholds, soft thresholds, ratios, schedules, rolling windows, stateful cooldowns, conditional logic, and composite expressions. A policy DSL or structured JSON format is compiled and validated before publication.

Core Pre-Trade Checks

- Requested risk and position size.
- Account buying power, cash, collateral, and margin.
- Maximum order notional and quantity.
- Daily, weekly, and rolling loss limits.
- Drawdown from high-water mark.
- Gross, net, directional, sector, asset-class, currency, and venue exposure.
- Instrument, issuer, token, chain, pool, and counterparty concentration.
- Leverage, delta, gamma, vega, duration, and other supported sensitivities.
- Correlation and factor exposure.
- Liquidity, spread, depth, expected slippage, and participation rate.
- Volatility and event-risk adjustments.
- Open-order exposure and pending settlement.
- Strategy and playbook eligibility.
- Behavioral controls such as revenge-trade cooldown or rule-violation state.
- Data freshness, provider quorum, and model-confidence requirements.
- Kill switch and incident state.

Position Sizing

The sizing engine supports fixed quantity, fixed notional, percentage of equity, fixed fractional risk, volatility targeting, stop-distance sizing, Kelly-derived fractions with policy caps, portfolio risk budgeting, and strategy allocation. A sizing result includes raw size, rounded venue size, estimated loss at invalidation, notional, margin, fees, slippage, confidence, and binding constraints.

Representative fixed-fractional calculation:

```
risk_budget = eligible_equity * risk_fraction * scaling_factor
unit_risk = abs(entry_price - invalidation_price) * contract_multiplier
raw_quantity = risk_budget / (unit_risk + estimated_cost_per_unit)
approved_quantity = round_down_to_lot(min(raw_quantity, all_policy_caps))
```

All inputs and rounding rules are stored with the evaluation.

Continuous Risk

Continuous risk recomputes exposure when positions, prices, volatility, liquidity, account balances, provider quality, policy, or incidents change. It can generate warnings, reduce permitted new exposure, cancel unfilled orders, request approval, trigger hedges, or activate kill switches according to policy.

Portfolio and Scenario Analytics

The analytics engine shall support:

- Realized and unrealized profit and loss with currency conversion.
- R-multiple, expectancy, win rate, payoff ratio, profit factor, drawdown, recovery factor, and return distribution.
- Time-weighted and money-weighted returns where inputs permit.
- Volatility, downside deviation, Sharpe-like and Sortino-like ratios, and tail loss measures.
- Exposure and attribution by strategy, instrument, sector, asset class, currency, venue, chain, and factor.
- Correlation matrices and concentration measures.
- Historical, parametric, and scenario-based stress tests.
- Gap, volatility shock, liquidity shock, correlation shift, provider failure, chain congestion, and depeg scenarios.
- Risk-of-ruin and capital survival simulations under explicit assumptions.

Metrics are definitions with versioned formulas, not ad hoc SQL fragments. Reports show sample size, missing-data treatment, time horizon, currency, benchmark, and confidence where relevant.

Trade and Process Analytics

Trade quality is separated from outcome. A profitable rule violation can receive a low process grade; a losing but compliant trade can receive a high process grade. Grade dimensions include preparation, setup quality, entry, size, stop placement, management, exit, documentation, emotional control, and rule adherence.

The analytics service calculates maximum favorable excursion, maximum adverse excursion, capture efficiency, hold-time efficiency, stop efficiency, target efficiency, entry timing, exit timing, and execution cost. Segmentation supports setup, regime, session, weekday, instrument, direction, confidence, emotion, account, and coach-defined cohorts.

Backtesting and Simulation

Backtests are reproducible jobs with immutable manifests. The system records strategy version, data sources, universe, dates, timezone, corporate-action handling, costs, slippage, latency assumptions, position sizing, risk policy, random seeds, code commit, container image, and result checksums.

The backtesting engine shall prevent look-ahead through explicit event-time sequencing and data availability timestamps. It shall support walk-forward validation, out-of-sample periods, parameter sensitivity, Monte Carlo reshuffling, bootstrap confidence intervals, and comparison to simple baselines.

Probability and Calibration

Probability outputs include target event, horizon, population, feature version, model version, raw probability, calibrated probability, uncertainty, and evidence. Calibration is measured over time and by segment. The interface avoids presenting an unqualified percentage without sample size, calibration status, and data freshness.

Overrides and Accountability

Risk policies can define whether a rule is non-overridable, approval-overridable, or administrator-configurable. The build does not hard-code a universal prohibition on override, but every override must be authorized, time-bounded, resource-scoped, reasoned, and auditable. Automated systems cannot self-approve an exception unless a published tenant policy explicitly assigns that authority and separation requirements are satisfied.

AI and Agent Architecture

Agent Platform Goals

The agent platform converts language models and specialized models into controlled, observable application workers. Agents do not own canonical business data and do not bypass domain services. They receive a task, context, tenant policy, and tool allowance; they produce validated structured outputs, tool calls, evidence references, and a complete run record.

The architecture supports multiple model vendors, self-hosted models, specialized forecasting or vision models, and deterministic functions. Model selection is a routing decision based on task capability, quality, latency, cost, data classification, region, and availability.

Agent Catalog

Agent	Primary responsibilities	Authorized outputs or tools
Trade analyst	Analyze trade plan, execution, context, and outcome	Analysis report, rule observations, grade proposal, evidence links
Market briefing agent	Assemble market-open and continuous intelligence	Briefing sections, scenarios, watchlist candidates, alerts
Risk officer agent	Explain risk results and identify mitigation options	Plain-language explanation, alternative sizing, approval draft
Trading coach	Detect behavioral patterns and manage improvement plans	Coaching observations, goal proposals, review prompts
Strategy researcher	Generate and test hypotheses using approved datasets	Research plan, experiment request, result synthesis
Screenshot analyst	Extract chart, order, annotation, and text information	Structured visual extraction with confidence and regions
Replay analyst	Reconstruct decision sequence and alternatives	Timeline, information set, counterfactual questions
Probability analyst	Build or interpret calibrated scenario estimates	Prediction request, explanation, calibration summary
Execution assistant	Prepare or manage policy-authorized order workflows	Order intent, order plan, cancel or replace proposal, command invocation
Portfolio analyst	Explain exposure, attribution, and stress results	Portfolio report, rebalance proposal, scenario request
Liquidity analyst	Evaluate pools, routes, depth, providers, and anomalies	Route comparison, liquidity alert, evidence set
Provider quality agent	Investigate provider failures and divergence	Diagnostic plan, incident update, runbook proposal
Incident assistant	Summarize timelines and coordinate response	Incident brief, action list, stakeholder update, postmortem draft
Compliance evidence agent	Assemble records and identify missing evidence	Evidence package request, gap report, export manifest

Agent Definition

An agent version includes:

- Purpose, owner, status, and allowed tenant data classes.
- System instructions and task templates.
- Input and output JSON schemas.

- Tool allowlist with per-tool conditions and maximum call counts.
- Model routing requirements and fallback policy.
- Retrieval collections and maximum context budget.
- Memory scope, expiry, and write permissions.
- Time, token, cost, and concurrency limits.
- Confidence and escalation rules.
- Evaluation suite and minimum release score.
- Required human review or policy authorization for material actions.

Published versions are immutable. A run always references the exact agent, prompt, tool, model, policy, and knowledge versions used.

Agent Execution Lifecycle

1. The application creates an agent task with tenant, environment, actor, purpose, resource references, and authorization context.
2. The agent gateway classifies data and selects an eligible model endpoint.
3. Context builders retrieve canonical records and tenant-scoped knowledge using explicit queries.
4. The model receives instructions, schemas, evidence, and tool descriptions.
5. Tool requests are validated against permission, schema, rate, and policy.
6. Tool results are sanitized and appended to the run transcript.
7. The final output is validated against its schema and business constraints.
8. A renderer converts the structured output to UI, report, or notification form.
9. Feedback and downstream outcomes are linked to the run.
10. Evaluation jobs sample or score the run for quality, policy compliance, latency, and cost.

Structured Output Contract

Agent output shall separate conclusions, evidence, uncertainty, and proposed actions. A representative analysis schema is:

```
{
  "summary": "The trade followed the planned entry but exceeded the approved size.",
  "confidence": 0.86,
  "observations": [
    {
      "code": "SIZE_POLICY_DEVIATION",
      "severity": "high",
      "claim": "Filled quantity exceeded the approved quantity by 18 percent.",
      "evidence_refs": ["risk:rsk_...", "execution:exe_..."]
    }
  ],
  "uncertainties": [
    {
      "issue": "The screenshot timestamp is ambiguous.",
      "impact": "Entry-context assessment may be incomplete."
    }
  ],
  "proposed_actions": [
    {
      "type": "create_plan_item",
      "title": "Use order quantity lock for the next five sessions",
      "requires_confirmation": true
    }
  ]
}
```

Free-form text may be rendered from this structure, but the structure is authoritative.

Tool Registry and Permissions

Tools are typed application capabilities such as `get_trade`, `query_market_bars`, `evaluate_risk`, `create_order_intent`, `submit_order`, `cancel_order`, `run_backtest`, `query_liquidity`, `open_incident`, or `generate_evidence_package`. Each tool declares input schema, output schema, data classification, permission code, environment support, idempotency behavior, and side-effect level.

Side-effect levels are:

- `read`: no canonical mutation.
- `draft`: creates a non-effective draft.
- `workflow`: opens a review, approval, or incident workflow.
- `financial`: can create, modify, cancel, or settle financial exposure.
- `privileged`: changes security, policy, provider, or platform configuration.

Agent policy determines which side-effect levels are available and whether confirmation or approval is required.

Model Router

The model router evaluates capability, context length, structured-output reliability, tool support, image support, latency target, cost ceiling, regional processing, tenant preference, and endpoint health. It can use cascades such as a low-cost classifier followed by a higher-capability model only when needed. Fallbacks are explicit and visible in the run record.

Model endpoints maintain health, error, latency, token, cost, and quality metrics. A degraded endpoint can be removed from routing without changing agent definitions.

Retrieval and Knowledge

Retrieval is tenant-scoped and policy-aware. Documents are parsed into versioned chunks with source, permissions, timestamps, and checksums. Query results are filtered before ranking, not only after retrieval. Embeddings do not serve as authorization.

Knowledge collections include tenant playbooks, policies, prior reviews, research, runbooks, instrument notes, integration documentation, and approved market references. Generated conclusions cite canonical resource identifiers or document chunks.

Memory

Memory is explicit rather than an unbounded transcript. Memory types include user preferences, tenant conventions, workflow state, accepted coaching goals, and stable integration mappings. Each item has scope, source, confidence, owner, expiry, and deleteability. Sensitive or high-impact facts require confirmation before persistence.

Vision and Screenshot Intelligence

Screenshot processing supports chart images, broker interfaces, order tickets, performance reports, and annotated notes. The pipeline stores the original file, image hash, metadata, extracted text, regions, chart features, symbol candidates, timeframe candidates, and confidence. Extracted values never replace canonical broker or market records without user confirmation or corroboration.

Agent Evaluation

Evaluation covers:

- Schema validity and tool-call correctness.
- Factual consistency with provided evidence.
- Citation and provenance completeness.
- Risk and policy compliance.

- Calibration of confidence.
- Hallucination, omission, and unsafe action rates.
- Task-specific quality judged by labeled examples or expert rubrics.
- Latency, cost, and fallback behavior.
- Performance by tenant, language, market, instrument, and model version.

Release requires offline suite success, adversarial tests, shadow traffic where possible, and monitored canary deployment. Online feedback cannot be the only evidence of correctness.

Prompt and Model Change Management

Prompt, model, tool, and retrieval changes are treated as production code. They require versioning, tests, review, evaluation comparison, rollback, and deployment records. A prompt hotfix may be expedited, but it still creates a new immutable version and evidence record.

Data Protection in AI Workloads

Before model invocation, the gateway classifies content and applies tenant policy for masking, redaction, endpoint eligibility, retention, and training use. Provider requests use enterprise or configured data-handling controls. Secrets, raw authentication material, and unrelated tenant data are excluded. Full prompts and responses are stored only when policy permits; otherwise the system stores hashes, structured results, and sanitized traces.

Agent Autonomy

The build supports all autonomy levels. Autonomy is granted through an explicit combination of agent version, tool permission, tenant policy, environment, resource scope, and actor delegation. An agent may move from analysis-only to draft, workflow, supervised financial, or autonomous financial operation without architectural change. The system always preserves the same authorization, risk, idempotency, execution, and audit path.

Codex Human-in-the-Loop Engineering Control Plane

Purpose and Control Boundary

The Codex human-in-the-loop control plane governs repository-changing work, command execution, and implementation evidence produced by coding agents. It is an engineering control plane, not a product feature gate. It shall not remove trading, analytics, execution, Hermes, integration, or AI capabilities from the architecture. Its purpose is to ensure that implementation work is specified, authorized, deterministic, auditable, bounded, and verifiable.

The runtime platform policy engine and the engineering control plane may share schemas, evaluation libraries, audit conventions, and approval concepts, but they remain separate trust domains. A development approval cannot authorize a production trade, and a trading approval cannot authorize a repository or infrastructure change. Cross-domain actions require independent authorization in each domain.

This paper is a final normative design baseline. It does not, by itself, assert that a software implementation has passed tests. The phrases **starter complete**, **release ready**, or **production ready** may be used only when the evidence required in this section has been generated from the exact commit being evaluated.

Mandatory Starter Standard

The repository shall include a runnable, production-oriented Codex starter that meets the following minimum standard. These requirements are additive to, and do not replace, the platform-wide security, testing, tenancy, risk, and deployment requirements elsewhere in this paper.

ID	Mandatory requirement	Minimum acceptance evidence
HITL-001	Spec Kit-compatible constitution, specification, plan, tasks, and acceptance checklist	Files parse, cross-reference correctly, and pass traceability validation
HITL-002	Repository-level AGENTS.md operating rules, with scoped overrides where necessary	Codex instruction discovery test and documented precedence
HITL-003	Deterministic TOML policy engine	Repeatable decisions for identical canonical inputs and policy version
HITL-004	Exact decisions allow, require_approval, and deny	Unit tests for each decision and conflict case
HITL-005	Expiring, hashed, request-scoped, single-use approval tokens	Expiry, replay, mismatch, and atomic-consumption tests
HITL-006	Validated workflow state machine	Invalid transitions rejected; all transitions persisted
HITL-007	SQLite audit and execution history for the local starter	Durable records, foreign keys, append-only controls, export test
HITL-008	Command execution through argument arrays with shell=False	Executor test proves no shell-string interpolation path exists
HITL-009	Bounded standard output, bounded standard error, and enforced timeouts	Truncation, timeout, termination, and result-metadata tests
HITL-010	Automated requirement-traceability validation	Non-zero exit on missing, duplicate, or dangling links
HITL-011	Minimum ten-test baseline plus project test suites	All ten baseline tests and all applicable project tests pass
HITL-012	GitHub Actions continuous integration	Clean run for tests, traceability, security, and container smoke checks

ID	Mandatory requirement	Minimum acceptance evidence
HITL-013	Reproducible Dockerfile	Non-root image builds and passes an offline verification smoke test
HITL-014	Security policy and operations runbook	SECURITY.md, incident contacts, supported versions, and recovery steps
HITL-015	Initialized Git repository with a baseline commit	Clean worktree, commit identifier, and verification report tied to commit
HITL-016	Single verification command	codex-hitl verify produces human-readable and machine-readable results
HITL-017	Recommended Codex sandbox and approval profile	Workspace-write mode with on-request approvals documented and tested
HITL-018	Production replacement path for local-only components	Documented adapters for enterprise identity, central audit, secrets, and sandboxing

The starter may contain additional capabilities. It shall not omit or weaken any requirement in this table without an approved architecture decision record, a documented compensating control, and an updated acceptance checklist.

Repository Artifact Contract

The starter shall use the following repository shape or an equivalent shape preserving the same responsibilities:

```
repository-root/  
  AGENTS.md  
  .specify/  
    memory/  
      constitution.md  
  specs/  
    000-codex-hitl-control-plane/  
      spec.md  
      plan.md  
      tasks.md  
      data-model.md  
      quickstart.md  
      contracts/  
      checklists/  
        requirements.md  
  policies/  
    default.toml  
    schema.json  
  tooling/  
    codex-hitl/  
      pyproject.toml  
      src/codex_hitl/  
        cli.py  
        config.py  
        policy.py  
        approvals.py  
        workflow.py  
        executor.py  
        audit.py  
        traceability.py  
        models.py  
    tests/  
      test_policy.py  
      test_approvals.py  
      test_workflow.py  
      test_executor.py  
      test_audit.py  
      test_traceability.py  
  scripts/
```



```

validate_traceability.py
docs/
  runbooks/
    codex-hitl-operations.md
  security/
    threat-model-codex-hitl.md
evidence/
  .gitkeep
.github/
  workflows/
    ci.yml
Dockerfile
SECURITY.md
README.md

```

In the unified monorepo, `tooling/codex-hitl` is the reference location. A standalone extraction may publish the same package independently, provided requirement identifiers, policy semantics, data migrations, test vectors, and evidence formats remain compatible.

Specification-Driven Delivery Hierarchy

Implementation shall follow the hierarchy below:

1. The constitution defines permanent engineering principles and non-negotiable controls.
2. This production paper defines platform-level scope and architecture.
3. A feature specification defines user outcomes, requirements, acceptance scenarios, and explicit exclusions.
4. The implementation plan defines architecture, dependencies, data design, migration, security, test strategy, and operational impact.
5. Tasks decompose the plan into independently verifiable work linked to requirement identifiers.
6. Code, migrations, policy, tests, documentation, and evidence implement those tasks.
7. The acceptance checklist verifies the result against requirements rather than against intent or narrative memory.

A code change that alters behavior shall update the affected specification, plan, task, test, or architecture decision in the same change set. Emergency remediation may temporarily precede full documentation only under the incident process; the documentation and traceability debt shall be recorded with an owner and expiry.

Constitution articles

The repository constitution shall include, at minimum, the following articles:

- **Specification before implementation.** Material behavior is defined before or with code, never only in an agent transcript.
- **Full capability without arbitrary architecture limits.** Product scope is not reduced through hidden hard-coded exclusions; operational exposure is controlled through policy.
- **Deterministic control before generative judgment.** Authorization, state transitions, risk limits, token validation, and command boundaries are deterministic.
- **Human authority for governed actions.** An approval is attributable to an authorized principal and cannot be invented or self-issued by the requesting agent.
- **Safe command execution.** Commands use structured argument arrays, controlled working directories, bounded resources, and recorded results.
- **Traceability.** Every mandatory requirement maps to tasks, implementation artifacts, tests, and acceptance evidence.
- **Evidence before completion.** Passing claims are generated from automated checks tied to a commit and policy hash.
- **Tenant, secret, and data isolation.** Development tooling cannot bypass platform privacy, tenancy, or credential controls.

- **Reversible delivery.** Changes include rollback or forward-recovery procedures proportional to impact.
- **No silent bypass.** Failures, truncation, degraded checks, exceptions, and overrides are visible in output and audit history.

Constitution amendments require a version increment, rationale, impact assessment, migration plan, and explicit approval by the designated engineering and security authorities.

AGENTS.md Operating Contract

The root AGENTS.md shall be concise, executable, and repository-specific. It shall tell Codex:

- The repository purpose, principal directories, and source-of-truth documents.
- Required setup, build, lint, type-check, migration, test, and verification commands.
- Which files are generated and must not be hand-edited.
- Dependency, architecture, schema, and API conventions.
- How requirement identifiers are added and referenced.
- Which operations require approval before execution.
- Which files or directories contain secrets, production state, or protected evidence.
- The definition of done, including `codex-hitl verify`.
- How to report blocked work, uncertainty, failed checks, and incomplete evidence.

Directory-specific AGENTS.md or AGENTS.override.md files may add stricter instructions for risk, execution, security, infrastructure, migrations, or Hermes collectors. A nested instruction file shall not silently weaken a root invariant. Any deliberate relaxation requires an approved policy or constitution change and shall be surfaced in the task plan.

The repository shall include an instruction-discovery verification that starts Codex or uses an equivalent inspection command to confirm which instruction files were loaded for representative working directories.

Deterministic TOML Policy Engine

Policy responsibilities

The engineering policy engine decides whether a requested action can execute immediately, requires explicit approval, or is denied. The evaluator shall be a pure deterministic function of:

- Canonical action request.
- Effective policy document and policy hash.
- Actor and role attributes supplied by a trusted identity source.
- Repository, branch, environment, and working-directory context.
- Requested executable, argument vector, file scopes, network needs, environment-variable names, timeout, and side-effect classification.
- Prior valid approval context, where applicable.

Model output, prose rationale, or tool-call text is not an authorization source. The language model may propose an action, but it cannot alter the canonical request after approval or reinterpret the policy result.

Decision semantics

The only terminal policy decisions are:

- `allow`: the exact canonical request may proceed without a separate approval token.
- `require_approval`: execution is suspended until an authorized approver issues a valid token for the exact request fingerprint.
- `deny`: execution shall not occur. An approval token cannot convert a deny into an allow.

The evaluator shall fail closed on invalid TOML, unknown decision values, missing required fields, unsupported schema versions, invalid condition types, or an unavailable policy. A failed-closed result is recorded as deny with a machine-readable reason code distinguishing configuration failure from an intentional rule denial.

Policy evaluation shall use a documented conflict algorithm. The reference algorithm collects all matching rules and applies deny over require_approval over allow. If multiple rules return the same decision, the evaluator records every matching rule identifier and selects the display rationale by highest priority, then lexical rule identifier. The default decision applies only when no rule matches.

Every result shall include:

```
{
  "decision": "require_approval",
  "reason_code": "NETWORK_OR_EXTERNAL_PATH",
  "matched_rule_ids": ["POL-120", "POL-310"],
  "policy_version": 7,
  "policy_sha256": "...",
  "request_sha256": "...",
  "evaluated_at": "2026-07-10T16:00:00Z"
}
```

Reference TOML policy

```
schema_version = 1
default_decision = "require_approval"

[[rules]]
id = "POL-100"
description = "Allow repository reads and deterministic verification"
decision = "allow"
priority = 100

[rules.match]
actions = ["read_file", "list_directory", "run_verification"]
network = false
workspace_only = true

[[rules]]
id = "POL-200"
description = "Require approval for dependency or migration changes"
decision = "require_approval"
priority = 200

[rules.match]
actions = ["install_dependency", "apply_migration", "git_commit"]
workspace_only = true

[[rules]]
id = "POL-900"
description = "Deny attempts to expose secrets or escape the approved workspace"
decision = "deny"
priority = 900

[rules.match]
secret_material = true
outside_workspace = true
```

The production policy schema shall support versioning, rule identifiers, descriptions, conditions, decision, priority, effective period, owner, change reference, and optional approval requirements. Policy documents shall be immutable after publication; a change creates a new version and hash.

Approval Requests and Tokens

Token properties

An approval token shall be:

- Generated from at least 256 bits of cryptographically secure randomness.
- Displayed only at issuance and never stored in plaintext.
- Stored as a keyed hash or equivalently secure verifier with key rotation support.
- Bound to the exact canonical request hash, policy hash, repository identity, branch or commit context, actor, approver, and permitted execution count.
- Time-limited with an explicit issued-at and expires-at timestamp.
- Single-use and consumed atomically in the same transaction that authorizes the workflow transition.
- Revocable before use and unusable after cancellation, request mutation, policy supersession where configured, or expiry.
- Compared using constant-time verification and redacted from logs, errors, traces, and command output.

The default local expiry should be short, such as fifteen minutes, but shall be policy-configurable. A token cannot authorize a request whose executable, arguments, working directory, environment names, file scope, network requirement, timeout, or side-effect level differs from the approved request.

Approval lifecycle

1. A validated request receives `require_approval`.
2. The system creates an approval request with a human-readable summary and canonical request hash.
3. An authorized approver reviews scope, policy reasons, expected effects, and rollback information.
4. Approval issuance records approver identity, reason, expiry, and token verifier.
5. The caller presents the plaintext token once.
6. The service verifies scope and expiry and atomically marks the token consumed.
7. The workflow advances to approved and then to execution.
8. Reuse, mismatch, expiry, or revocation is rejected and audited.

Separation-of-duties policy may require a different approver from the requester, two approvers, elevated authentication, or an external change ticket. The local starter shall implement one authorized approver while preserving a data model that supports multiple approvals.

Validated Workflow State Machine

Every controlled action shall follow a persisted state machine. The reference states are:

State	Meaning	Permitted next states
created	Request record exists but has not been validated	validated, failed, canceled
validated	Schema, repository, and command structure are valid	policy_evaluated, failed, canceled
policy_evaluated	Deterministic policy result is stored	approved, awaiting_approval, denied, failed
awaiting_approval	Exact request is suspended for a human decision	approved, denied, expired, canceled
approved	Policy or token authorizes execution	queued, canceled, expired
queued	Execution resources have been assigned	running, canceled, failed
running	Child process is active	succeeded, failed, timed_out, canceled
succeeded	Process completed with accepted result	terminal

State	Meaning	Permitted next states
failed	Validation, setup, process, or post-check failed	terminal or explicit retry workflow
timed_out	Deadline was exceeded and termination completed	terminal or explicit retry workflow
canceled	Authorized cancellation completed	terminal
denied	Policy or approver rejected the request	terminal
expired	Approval or execution window elapsed	terminal

Transitions shall be implemented through a single validator, not through scattered status assignments. Each transition records previous state, next state, sequence number, actor, timestamp, reason code, request version, and correlation identifier. Invalid transitions fail without mutating state and generate an audit event.

Optimistic concurrency or an equivalent transaction guard shall prevent two workers from consuming one approval or starting the same execution. Retry requires a new workflow attempt linked to the original request; it does not rewrite prior history.

SQLite Audit and Execution History

The standalone starter shall use SQLite so it is runnable without external services. SQLite is a local control-plane store, not the final multi-tenant production audit system. The schema shall include at least:

Table	Purpose
policy_evaluations	Canonical request, decision, reasons, policy version, and hashes
approval_requests	Approval state, requester, scope, policy result, and expiry
approval_tokens	Hashed verifier, issuance metadata, revocation, and atomic consumption state
workflow_runs	Current state, version, correlation, timestamps, and terminal result
workflow_transitions	Immutable ordered transition history
command_executions	Executable, arguments, working directory, bounds, exit, timing, and output references
audit_events	Append-only security and operational events
requirement_links	Requirement, task, source, test, and evidence relationships

The database shall enable foreign keys, transactional writes, an explicit busy timeout, and write-ahead logging when compatible with the deployment. Application roles shall not update or delete immutable history. The starter shall use triggers or an equivalent repository rule to reject unauthorized updates and deletes from append-only tables.

Audit records shall include event identifier, timestamp, actor, action, target, tenant or repository scope, request and policy hashes, workflow identifier, outcome, reason code, correlation identifier, and sanitized metadata. Raw approval tokens, secrets, full environment values, and unredacted sensitive output shall never be stored.

The production adapter shall write equivalent records to Supabase Postgres with row-level security, centralized retention, backup, export, legal-hold support, and SIEM delivery. Local and production schemas shall share stable event types and identifiers so evidence remains comparable.

Safe Command Executor

The reference executor shall be implemented in Python and invoke commands through an argument sequence with shell=False. There shall be no general path that accepts a free-form shell command string and forwards it to a system shell.

A representative invocation pattern is:

```
process = subprocess.Popen(
    argv,
    shell=False,
    cwd=validated_workspace,
    env=filtered_environment,
    stdin=subprocess.DEVNULL,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    start_new_session=True,
)
```

The final implementation shall additionally enforce:

- `argv` is a non-empty array of strings with bounded item and total length.
- The executable resolves through an approved path rule; ambiguous current-directory resolution is rejected.
- The working directory resolves inside the approved workspace unless policy explicitly authorizes another directory.
- Environment variables are assembled from a minimal allowlist; values are never copied wholesale from an interactive shell.
- Standard input is disabled unless a typed input channel is explicitly part of the approved request.
- Shell scripts may run only as versioned files invoked by an explicit interpreter and argument array; dynamically composed shell text is not accepted.
- File writes, network access, external paths, dependency installation, Git history mutation, migration application, and infrastructure changes are represented in policy context.
- Child processes run in their own process group so timeout or cancellation can terminate descendants.
- Platform-specific resource controls constrain CPU, memory, process count, open files, and disk use where available.
- Secrets and approval material are redacted before persistence or display.

A successful process exit does not automatically mean workflow success. Post-execution checks may verify expected files, checksums, tests, traceability, migration state, or repository cleanliness before the workflow enters succeeded.

Bounded Output and Timeout Behavior

The executor shall capture standard output and standard error independently without unbounded memory growth or pipe deadlock. The reference behavior is:

- Stream both pipes concurrently.
- Maintain a configurable per-stream byte cap and optional combined cap.
- Preserve the beginning and end of truncated output or write encrypted full output to an approved artifact store when policy permits.
- Record byte counts, truncation flags, encoding errors, and content hashes.
- Decode with an explicit error strategy; binary output is treated as an artifact rather than forced into logs.
- Enforce wall-clock timeout and optional inactivity timeout.
- On timeout or cancellation, send graceful termination to the process group, wait for a bounded grace period, then force termination.
- Record requested timeout, actual duration, termination signals, exit code, and whether descendants were confirmed stopped.

A timeout, truncation, decoding failure, or forced termination shall be visible in CLI output, audit records, and verification evidence. It shall never be silently converted into a clean success.

Codex Sandbox and Approval Operating Mode

The recommended interactive mode is:

```
codex --sandbox workspace-write --ask-for-approval on-request
```

This mode allows normal work inside the repository while surfacing actions that cross the configured sandbox boundary or require additional privileges. Network access remains disabled unless explicitly configured and approved. Read-only mode shall be available for planning and review. Full-access modes may be used only in isolated, disposable, organization-approved environments with external controls and audit; they are not the default development posture.

For managed deployments, organization policy shall constrain allowed sandbox modes, approval policies, network behavior, writable roots, and external tool connections. The repository policy engine supplements Codex sandboxing; it does not assume the sandbox alone provides business authorization or complete evidence.

Requirement Traceability Model

Every normative requirement shall have a stable identifier. Recommended prefixes include:

- BR for business requirements.
- FR for functional requirements.
- NFR for non-functional requirements.
- SEC for security and privacy requirements.
- RISK for trading and portfolio risk requirements.
- AI for agent and model requirements.
- HERMES for liquidity and provider requirements.
- OPS for operations and reliability requirements.
- HITL for Codex control-plane requirements.

The traceability graph shall support the following relationship chain:

```
paper requirement
-> feature specification requirement
-> implementation task
-> source or migration artifact
-> automated test
-> acceptance checklist item
-> generated evidence
-> release or commit identifier
```

The validator shall fail when:

- A mandatory requirement has no implementation task.
- A task has no requirement or approved maintenance reference.
- A critical requirement has no automated test.
- A referenced test, file, checklist item, or evidence artifact does not exist.
- Requirement identifiers are duplicated or malformed.
- A completed task has an unchecked acceptance criterion.
- Evidence was generated from a different commit, policy hash, or dependency lock than the release candidate.
- A protected source package changes without a linked task or approved emergency exception.

The traceability output shall be available as JSON and a readable Markdown or HTML report. A passing report includes counts for requirements, tasks, source links, tests, checklist items, evidence records, warnings, and failures.

CLI Contract

The starter shall provide a `codex-hitl` command with the following baseline interface:

```
codex-hitl init
codex-hitl verify [--json evidence/verification.json]
codex-hitl policy evaluate --request request.json
codex-hitl approval request --workflow <id>
```

```
codex-hitl approval grant --request <id> --expires-in 15m
codex-hitl run --request request.json --approval-token <token>
codex-hitl workflow show <id>
codex-hitl audit export --format jsonl --output evidence/audit.jsonl
```

init creates missing local configuration and database state without overwriting reviewed files. verify is read-only except for writing evidence output. Every subcommand shall have deterministic exit codes, structured error messages, and a --json mode suitable for CI.

The local developer sequence is:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e "tooling/codex-hitl[dev]"

codex-hitl init
codex-hitl verify
```

A standalone extraction may use python -m pip install -e .. The exact installation path shall be documented in the repository quickstart.

Ten-Test Baseline

The starter shall not be declared complete until the following ten automated tests pass. Projects may split these cases into more than ten test functions, but the evidence report shall map results back to these ten baseline cases.

Test ID	Required scenario	Passing condition
T-HITL-01	Allow decision	Matching safe request returns allow with stable hashes and reason codes
T-HITL-02	Approval decision	Governed request returns require_approval and cannot run without a token
T-HITL-03	Deny decision	Denied request never executes, including when a token is presented
T-HITL-04	Policy failure closes safely	Malformed or unsupported policy produces a recorded denial
T-HITL-05	Token expiry	Expired token is rejected without changing execution state
T-HITL-06	Token scope and single use	Mismatch and replay fail; one valid request consumes the token atomically
T-HITL-07	State transition validation	Invalid or concurrent transition is rejected and audited
T-HITL-08	Structured command execution	Executor uses shell=False, preserves argument boundaries, and enforces workspace scope
T-HITL-09	Output and timeout bounds	Excess output is bounded and a timed-out process group is terminated and recorded
T-HITL-10	Durable audit history	Evaluation, approval, transition, and execution history survive restart without secret leakage

The requirement-traceability validator is a separate mandatory check in addition to the ten tests. CI shall also run platform tests affected by the change, including type checks, linting, database tests, RLS tests, contract tests, and security scans where applicable.

GitHub Actions Continuous Integration

The baseline workflow shall run on pull requests and protected branches. It shall include:

1. Checkout with a pinned action version and minimal token permissions.
2. Pinned Python and Node runtimes as required by the monorepo.
3. Locked dependency installation and cache integrity checks.
4. Formatting, linting, static typing, and package-boundary validation.
5. The ten HITL baseline tests and all affected project tests.
6. `codex-hitl verify` and the standalone traceability check.
7. Secret, dependency, license, and static security scanning.
8. Docker image build and non-root smoke verification.
9. Evidence upload containing commit, policy hash, lockfile hashes, test results, and traceability report.

Production artifacts shall be built only by CI from a reviewed commit. Workflow permissions shall be least-privilege, untrusted pull requests shall not receive production secrets, and deployment jobs shall require protected environments and designated approvals.

Docker and Runtime Packaging

The starter Dockerfile shall:

- Pin the base image by version and, for release builds, by digest.
- Install only required runtime dependencies.
- Run as a non-root user.
- Copy dependency metadata before source to support reproducible caching.
- Avoid embedding credentials, approval tokens, development databases, or generated evidence.
- Expose a read-only verification or health command.
- Support a read-only root filesystem with explicit writable mounts for workspace, temporary data, and SQLite state.
- Define sane process, memory, and filesystem expectations for the deployment orchestrator.

The container smoke test shall initialize an ephemeral database, load the default policy, run `codex-hitl verify`, and exit non-zero on any failure.

Security Policy, Threat Model, and Operations Runbook

`SECURITY.md` shall state supported versions, vulnerability reporting channels, response expectations, disclosure process, secret-handling rules, and the boundary between the local starter and an enterprise deployment.

The threat model shall cover at least:

- Prompt or instruction injection attempting to alter policy context.
- Shell injection and argument confusion.
- Workspace traversal and symlink escape.
- Approval-token theft, leakage, replay, race, or scope substitution.
- Policy tampering, downgrade, ambiguity, or hash mismatch.
- Audit deletion, modification, truncation, or rollback.
- Secret exposure through environment, output, exceptions, traces, or artifacts.
- Malicious dependencies, build scripts, Git hooks, and generated files.
- Denial of service through output volume, process trees, timeouts, files, or database locks.
- Confused-deputy behavior between Codex, the local CLI, CI, and production systems.

The operations runbook shall include initialization, backup, restore, policy publication, token-key rotation, audit export, incident triage, database recovery, stuck-workflow recovery, forced process termination, release rollback, evidence generation, and escalation ownership.

Git Baseline and Change Integrity

The repository shall be initialized with Git and contain a baseline commit after the starter files, tests, policy, documentation, and verification configuration are complete. The baseline evidence shall record:

- Commit identifier and branch.
- Clean or explicitly documented worktree state.
- Dependency-lock hashes.
- Policy and constitution hashes.
- Database migration level.
- Test and traceability results.
- Container image identifier when built.
- Verification tool version.

Subsequent completion reports shall reference a commit, not an uncommitted directory. Signed commits and signed tags should be used when organization identity and key management are available.

Verification Output and Completion Standard

`codex-hitl verify` shall evaluate, at minimum:

- Required files and directory structure.
- Constitution, specification, plan, tasks, and checklist consistency.
- Policy parsing, schema, version, and deterministic test vectors.
- SQLite migration and append-only behavior.
- Approval-token configuration and key availability without exposing key material.
- Workflow state-machine coverage.
- Command-executor configuration, output bounds, and timeout defaults.
- Ten-test baseline status.
- Requirement-traceability status.
- Security policy, threat model, runbook, CI workflow, and Dockerfile presence.
- Git repository, baseline or current commit, and worktree state.

The command shall return zero only when all mandatory checks pass. Warnings shall not be hidden; policy determines whether a warning is release-blocking. Machine-readable output shall contain tool version, timestamp, commit, policy hash, individual checks, test references, warnings, failures, and overall status.

A valid starter completion statement must be generated from this evidence and shall include:

- The exact commit verified.
- The number of tests collected, passed, failed, skipped, and expected.
- A separate traceability result.
- The policy hash and database schema version.
- The container smoke result.
- Any enterprise production gaps still open.

No manually written statement that merely says tests passed is sufficient release evidence.

Enterprise Production Delta

The local starter is complete for repository-level development control when the preceding acceptance standard passes. Enterprise production deployment additionally requires organization-specific implementation of:

- Central identity, SSO, service identity, and approver group synchronization.
- Centralized, durable, access-controlled audit retention and SIEM delivery.
- Managed secrets, token-verifier keys, signing keys, and rotation procedures.
- Signed policy bundles, controlled publication, rollback, and policy-distribution integrity.
- Per-job or per-tenant sandboxing with operating-system, container, or virtual-machine isolation.
- Controlled network egress, private package registries, dependency provenance, and artifact signing.
- High-availability workflow coordination and transactional approval consumption.
- Backup, disaster recovery, legal hold, retention, privacy, and regional residency controls.
- Multi-party approval and separation of duties for infrastructure, production data, and financial-action code.
- Deployment integration with the platform's Supabase, observability, incident, and evidence services.

These enterprise requirements are deployment obligations, not reasons to omit the local starter. The starter shall expose interfaces and migration paths so the local SQLite, local identity, and local key implementations can be replaced without changing policy decisions, workflow semantics, requirement identifiers, or evidence contracts.

External Implementation Alignment

At publication on July 10, 2026, the implementation baseline is aligned with the official OpenAI Codex guidance for workspace sandboxing, on-request approvals, network controls, and repository-level AGENTS.md instructions. It is also aligned with GitHub Spec Kit's specification-driven flow of specification, plan, tasks, and implementation.

Spec Kit shall be installed only from the official `github/spec-kit` repository and pinned to a reviewed release tag. The current release verified for this paper is `v0.12.9`; future upgrades require compatibility tests and an updated toolchain manifest rather than an unpinned install.

Market Intelligence and Provider Quality

Market Data Architecture

Market data enters through provider adapters and is normalized into canonical instruments, timestamps, units, and event types. The platform distinguishes streaming data, request-response data, reference data, event calendars, and unstructured intelligence. Provider-specific payloads are retained only as allowed and are linked through content hashes or encrypted raw archives.

Supported Data Families

- Instrument and venue reference data.
- Quotes, trades, bars, order books, and auction data.
- Options chains, implied volatility, greeks, open interest, and volume.
- Futures curves, rolls, settlement, and open interest.
- Foreign-exchange rates and sessions.
- Digital-asset prices, funding, open interest, order books, and on-chain context.
- Economic calendars, corporate actions, earnings, and scheduled events.
- News, filings, announcements, and tenant-approved research.
- Chain blocks, logs, pool state, oracle observations, gas, and route quotes.

The schema is extensible so unsupported data types can be added without changing unrelated domains.

Provider Quality Dimensions

Each provider and endpoint is measured across:

Dimension	Definition
Availability	Successful responses or stream continuity during expected service windows
Freshness	Difference between expected current time or block state and observation time
Latency	Request, stream, and end-to-end ingestion delay
Completeness	Presence of required fields, symbols, intervals, and expected records
Correctness	Consistency with trusted references, venue records, or later reconciliation
Divergence	Difference from peer-provider observations after normalization
Stability	Error bursts, schema changes, throttling, disconnects, and variance
Coverage	Markets, instruments, fields, history, and network support
Cost efficiency	Cost per useful request, record, or decision under policy
Operational support	Incident communication, status visibility, and recovery behavior

Scores are time-windowed and segmented by capability, region, instrument, network, and endpoint. A provider can be healthy for one capability and unhealthy for another.

Canonicalization and Symbol Mapping

Canonical instruments use stable internal identifiers. Provider symbols are mappings with effective dates, venue, currency, contract details, and confidence. Ambiguous mappings enter a review queue. Corporate actions, contract rolls, token migrations, redenominations, and chain-specific addresses are represented as events, not destructive edits.

Freshness and Confidence

Every market-facing API response includes source, observed time, ingested time, freshness status, and confidence. A query policy specifies maximum age, minimum provider count, maximum divergence, and fallback behavior. Consumers can require `strict`, `standard`, or `best_available` quality profiles, or define a custom profile.

Provider Quorum

A quorum request gathers observations from eligible providers, normalizes them, removes malformed or stale values, calculates divergence, and applies a decision rule. Quorum methods may include median, trimmed mean, weighted median, majority agreement, threshold consensus, or source-priority fallback.

A quorum decision records:

- Query and normalization version.
- Required and responding providers.
- Included, excluded, and missing observations.
- Freshness and latency for each observation.
- Divergence statistics and thresholds.
- Provider weights and current quality scores.
- Canonical value or explicit no-decision result.
- Confidence, reason codes, and expiry.

No-decision is a valid outcome and must be handled by downstream policy.

Data Storage and Downsampling

High-frequency raw data may be retained for a configurable window, then downsampled into bars, statistical summaries, or compressed archives. The system preserves manifests required to reproduce analytics. Tenant-generated annotations and decisions retain references to the exact data snapshot or query parameters used.

Market Briefing Pipeline

The briefing pipeline uses schedules and market calendars to assemble:

1. Prior-session and overnight movement.
2. Volatility, liquidity, breadth, and regime state.
3. Scheduled economic, corporate, protocol, and chain events.
4. Major news and tenant-relevant research.
5. Open positions, concentration, and risk state.
6. Watchlist candidates and playbook matches.
7. Scenario trees with triggers, invalidations, and evidence.
8. Data-quality notes and known provider incidents.

The agent writes the narrative, but deterministic services provide measurements and event lists. Users can drill into every claim.

Smart Watchlists and Alerts

Watchlists combine explicit filters, scoring functions, model outputs, portfolio context, and user preferences. Each candidate includes reason codes, freshness, rank components, and disqualifiers. Alerts are deduplicated, severity-rated, and stateful; they can open, acknowledge, resolve, suppress, or escalate.

Regime Detection

Regime models may classify trend, mean reversion, volatility, correlation, liquidity, risk appetite, or chain conditions. Results include probabilities rather than only hard labels. Models are evaluated for transition stability, economic usefulness, calibration, and segment performance. Regime output is a feature for policy and analysis, not a hidden replacement for risk calculation.

Hermes Protocol and Multi-Chain Liquidity Intelligence

Purpose

Hermes provides durable, explainable intelligence for multi-chain liquidity and the infrastructure used to observe it. It treats provider quality, chain finality, token identity, pool state, route economics, and operational incidents as first-class domains rather than incidental API responses.

Chain and Network Model

A chain registry defines network identifier, environment, consensus and finality metadata, native asset, address format, block cadence expectations, reorganization handling, supported transaction simulation, and explorer links. Configuration is versioned because network properties and endpoints change.

The build supports account-based and UTXO-like networks, EVM and non-EVM chains, layer-two networks, rollups, sidechains, app chains, and additional architectures through adapter capabilities. Chain-specific logic is isolated behind typed interfaces.

Provider Types

Hermes manages:

- RPC and archival node providers.
- Indexers and data APIs.
- Price and reference oracles.
- DEX and aggregator quote providers.
- Transaction simulation providers.
- Block, mempool, and gas data providers.
- Token metadata and security-data providers.
- Bridge and cross-chain messaging providers.

A provider profile can contain multiple endpoints by region, network, capability, and credential reference.

Collection and Observation Pipeline

Collectors run scheduled and event-driven probes. A probe definition specifies network, capability, request, expected schema, timeout, retry, and validation. Raw responses are hashed and optionally retained. Normalized observations include provider, endpoint, block context, event time, ingestion time, latency, error class, and validation results.

Collectors must handle rate limiting, pagination, websocket reconnects, chain stalls, reorgs, inconsistent decimals, token metadata changes, and provider schema drift. A collector cannot silently coerce invalid data into a valid observation.

Finality and Reorganization Handling

Observations have finality states such as pending, safe, finalized, orphaned, or network-specific equivalents. The platform tracks block lineage and detects when a previously observed block is no longer canonical. Derived pool, route, alert, or decision records reference block context and can be invalidated or recomputed after a reorganization.

Token Identity and Validation

A canonical token may have contracts on multiple networks. Token-contract records include network, address, decimals, symbol, name, implementation metadata, effective dates, verification status, and risk annotations. Symbol and name are never sufficient identity. Conflicts, proxies, rebasing behavior, transfer fees, blacklists, pauses, wrappers, and migration events are represented when known.

Liquidity Venue and Pool Model

Venues include automated market makers, order books, RFQ systems, aggregators, bridges, and hybrid mechanisms. Pool records include protocol, network, contract or market identifier, token set, fee model, curve or invariant type, deployment block, status, and adapter version.

Pool snapshots may contain reserves, virtual reserves, active liquidity, tick state, balances, fees, volume, transactions, spread, mid-price, depth at configured basis-point bands, and expected price impact for standard notionals. Unsupported fields remain null with capability metadata; they are not fabricated.

Route Intelligence

A route request specifies source asset, destination asset, amount, network constraints, slippage tolerance, time horizon, bridge policy, and execution preferences. Candidate routes are evaluated for:

- Expected output and minimum output.
- Price impact and depth.
- Venue and provider fees.
- Gas and bridge costs.
- Estimated latency and finality.
- Number and type of legs.
- Provider and pool health.
- Token behavior and transfer constraints.
- Simulation result and revert risk.
- Cross-chain settlement and bridge exposure.
- Data freshness and quorum confidence.

Route results include all assumptions and expire when block, quote, gas, or pool conditions move beyond policy tolerance.

Liquidity Anomalies

Anomaly rules and models detect:

- Sudden depth or reserve loss.
- Spread or price-impact expansion.
- Cross-provider or cross-venue price divergence.
- Oracle lag or disagreement.
- Pool imbalance, unusual volume, or fee changes.
- Chain stall, finality delay, congestion, or gas shock.
- Repeated quote failures or transaction simulation reverts.
- Token metadata or contract behavior changes.
- Provider error, schema drift, or endpoint regional failure.

Alerts contain severity, affected resources, baseline, observed change, evidence, confidence, and recommended operator actions.

Provider Quality and Quorum Operations

Operators can add, test, enable, disable, weight, or quarantine providers. Changes are versioned, permissioned, and audited. Provider score is not a single global number; it is a matrix by network, capability, endpoint, region, and time window.

Quorum policies define minimum providers, eligible tiers, freshness, divergence, weighting, fallback, and no-decision behavior. Critical execution or valuation workflows can require stronger quorum than dashboard display.

Operator Dashboard

The Hermes operator interface includes:

- Global and tenant-specific provider health.
- Network status, block freshness, finality, and reorganization indicators.
- Endpoint latency and error heat maps.
- Quorum pass, degraded, and no-decision rates.
- Pool and route liquidity summaries.
- Active anomalies, incidents, and runbook status.
- Provider configuration changes and deployment history.
- Evidence package generation and audit search.

Every chart links to raw or normalized observations and query definitions.

Runbooks and Automated Remediation

Runbooks are versioned workflows with preconditions, tool steps, approvals, rollback, timeout, and evidence. Examples include endpoint rotation, provider weight reduction, cache invalidation, collector restart, chain backfill, quarantine, route suppression, or tenant notification. Runbooks may be manual, supervised, or policy-authorized automated actions.

Hermes APIs

Hermes APIs expose provider registry, health samples, scores, chain state, token and pool data, quorum requests, route requests, anomalies, incidents, and evidence. High-volume streams use subscriptions with sequence numbers and replay. All responses include network and block context.

Hermes Production Evidence

A production evidence package includes RLS and authorization tests, provider contract tests, quorum scenario tests, chain reorg tests, incident simulations, runbook outcomes, deployment records, health history, backup and restore evidence, and current configuration checksums.

User Experience and Interface Architecture

Experience Principles

The interface shall prioritize decisions, evidence, and workflow state. It must not conceal uncertainty behind polished narrative. Every score, alert, agent conclusion, risk decision, and route result exposes freshness, provenance, assumptions, and drill-down evidence.

The design system uses accessible components, consistent density controls, keyboard navigation, responsive layouts, and a command palette. Users can switch between simplified and professional views without changing the underlying data.

Information Architecture

Primary navigation groups are:

- **Home:** role-specific command center and active work.
- **Trade:** accounts, orders, positions, journal, replay, and execution.
- **Plan:** briefing, watchlists, playbooks, strategies, and research.
- **Risk:** exposure, limits, breaches, approvals, scenarios, and kill switches.
- **Coach:** observations, goals, reviews, behavioral patterns, and progress.

- **Analytics:** performance, process, execution quality, backtests, and reports.
- **Hermes:** providers, chains, pools, routes, quorum, alerts, incidents, and runbooks.
- **Operations:** jobs, integrations, notifications, evidence, and audit.
- **Administration:** members, roles, environments, policy, security, data, and billing hooks.

Navigation is generated from capability and permission manifests.

Trader Command Center

The trader home view shows current session, account and portfolio state, daily risk budget, active positions and orders, market regime, event risks, watchlist, open coaching actions, unresolved journal items, and system health. Widgets are configurable and retain their data source and freshness.

A pre-trade ticket combines thesis, playbook, entry, invalidation, targets, quantity, expected risk, market data, risk result, and execution route. A user can save a draft, request approval, simulate, paper trade, or submit live according to policy.

Journal and Replay Experience

The journal provides timeline, table, and calendar views. A trade detail page combines execution facts, charts, screenshots, notes, emotions, plan, grade, analysis, and linked coaching items. Replay uses synchronized chart, market events, order events, notes, and available information. Users can hide future data to preserve review integrity.

Risk Console

The risk console includes portfolio exposure, limit utilization, stress, breaches, pending approvals, overrides, and kill switches. Operators can trace any result to measurements, policy rules, source data, and effective versions. Bulk actions require explicit scope and confirmation.

Coach Experience

The coach interface presents trends, repeated violations, strengths, emotional triggers, goal progress, review queues, and evidence-backed suggested interventions. Coaches can compare performance across time or cohorts only within authorized scope. Comments and plan changes remain linked to the reviewed records.

Hermes Operations Experience

The Hermes interface emphasizes live health, divergence, incidents, and evidence. Provider and network pages combine status, history, recent changes, test results, and affected decisions. Quorum and route pages show included and excluded observations and support replay from historical data.

Notifications and Work Inbox

A unified inbox contains approvals, risk warnings, journal tasks, coach reviews, provider incidents, failed jobs, security notices, and evidence requests. Items have status, owner, priority, due date, resource link, and escalation policy. Notification channels are preferences layered under mandatory security and incident policies.

Explainability Patterns

The design system includes standard components for:

- Confidence and uncertainty.
- Data freshness and source provenance.
- Policy decision and rule breakdown.
- Model and agent version.
- Provider quorum composition.

- Before and after comparison.
- Audit timeline.
- Approval and override chain.
- Reconciliation status.
- Simulation versus live environment identity.

Accessibility and Localization

The product targets WCAG-aligned keyboard, screen-reader, contrast, focus, form, and motion behavior. Charts provide textual summaries and data tables. Timezone, date, number, and currency formatting follow user and tenant preferences while canonical APIs remain explicit. The architecture supports translated interface strings and agent output language selection.

Mobile and Responsive Behavior

Mobile views support monitoring, alerts, approvals, journaling, notes, screenshots, and emergency controls. Complex analysis and policy editing may use desktop-optimized layouts, but no critical incident or execution action may require a desktop-only path without an alternative.

UI State and Data Fetching

Client state is limited to presentation and draft interaction. Server state is fetched through typed query clients with cache keys including tenant and environment. Optimistic updates are used only for reversible low-risk actions. Financial and privileged actions display committed server state and version.

Security, Identity, Privacy, and Tenant Isolation

Security Objectives

The security program protects confidentiality, integrity, availability, tenant isolation, financial action authority, and evidentiary reliability. Security controls are designed into the domain and deployment architecture rather than added after feature completion.

Authentication

Supabase Auth provides the initial user identity layer with email and OAuth support. The production design also supports multi-factor authentication, WebAuthn or passkeys, enterprise SSO through OIDC or SAML integration, session revocation, device and authentication-event history, and step-up authentication for high-risk actions.

Service identities use short-lived credentials, workload identity, or scoped API keys. Human accounts are not reused for background services.

Authorization

Authorization is enforced at multiple layers:

1. Route and UI capability checks for user experience.
2. API permission and ABAC evaluation before command handling.
3. Domain-level invariant and policy validation.
4. Postgres RLS for tenant and resource access.
5. Storage policies for tenant-owned files.
6. Event and worker context verification.
7. Integration and tool-specific permissions.

A denial at any layer stops the action. Platform-admin access uses separate roles, step-up authentication, explicit reason, limited duration, and immutable audit.

Tenant Isolation

Tenant isolation shall be tested with real database policies, not mocked query filters. Test suites attempt cross-tenant select, insert, update, delete, function invocation, storage access, vector retrieval, job processing, cache collision, realtime subscription, export, and audit search.

Tenant identifiers are included in database primary or unique indexes where necessary to prevent cross-tenant key collisions. Cache and object-storage keys are namespaced. Queue consumers verify the tenant context on every message. Analytics pipelines retain tenant labels and access controls.

Secrets and Credentials

Broker, exchange, provider, model, and webhook secrets are stored in a dedicated secrets manager or encrypted vault. Application tables store only opaque references and metadata. Access is issued to authorized runtimes for the shortest practical duration. Secrets are rotated, usage is audited, and revoked credentials invalidate active connectors.

Credentials shall not be exposed to the browser, AI prompt, log, trace, exception, support export, or test fixture. Local development uses synthetic or sandbox credentials through environment tooling.

Encryption and Key Management

Data is encrypted in transit and at rest. Highly sensitive fields may use application-level envelope encryption with tenant or data-class keys. Key rotation and re-encryption are supported. Export packages may be encrypted with tenant-provided public keys or time-limited download keys.

Application Security

Required controls include secure headers, CSRF protection where applicable, input validation, output encoding, dependency and container scanning, software bill of materials, signed builds, branch protection, secret scanning, rate controls, bot and abuse detection, file-type and malware scanning, and restrictive content security policy.

File uploads are quarantined until validated. Image and document processing runs in isolated workers with resource and network limits.

Financial Action Security

Financial actions require stronger protections:

- Step-up authentication or recent trusted authentication according to policy.
- Explicit account and environment identification.
- Idempotency and replay protection.
- Risk evaluation and policy version binding.
- Approval separation where configured.
- Signed or integrity-protected internal command messages.
- Dedicated execution network paths and service identities.
- Kill switches and anomaly monitoring.
- Reconciliation and uncertain-state handling.

AI Security

The agent gateway defends against prompt injection, tool misuse, data exfiltration, and poisoned retrieval. Retrieved content is treated as untrusted data, not instructions. Tool calls require schema and authorization validation. Agent output is not executed as code. URLs, files, and external content are fetched through controlled services with allowlists, scanning, and resource limits.

Privacy and Data Classification

Data classes include public reference, tenant internal, confidential trading, personal, authentication, secret, and regulated or contract-restricted data. Classification determines storage, access, model eligibility, logging, retention, export, and deletion rules.

Users can inspect connected data sources, model processing settings, memory items, exports, and deletion status. Privacy requests are workflow-driven and produce evidence without exposing unrelated tenant information.

Security Logging and Detection

Security events include authentication changes, privilege changes, secret access, connector changes, policy publication, execution-mode changes, failed authorization, cross-tenant attempts, unusual API access, export generation, kill-switch changes, and agent attempts to call denied tools. Alerts are correlated with deployment and incident records.

Threat Modeling

Threat models are maintained for authentication, tenancy, broker connections, order execution, webhooks, file processing, AI agents, retrieval, Hermes providers, chain data, admin support, and CI/CD. Each model identifies assets, trust boundaries, threats, controls, test cases, owners, and residual risk.

Security Testing and Remediation

Security gates include static analysis, dependency scanning, infrastructure policy checks, API fuzzing, RLS tests, authorization tests, secret scans, penetration testing before major release, and remediation SLAs by severity. Exceptions require an owner, expiry, compensating controls, and approval.

Governance, Compliance, Audit, and Model Oversight

Governance Model

Governance is implemented as a set of versioned decisions, approvals, evidence, and ownership assignments. Product, engineering, security, risk, AI, data, and operations each have defined authority. No single team can unilaterally declare a high-impact capability production-ready without the required evidence and sign-offs.

A governance register tracks policies, models, agents, integrations, data providers, risk rules, runbooks, deployments, incidents, exceptions, and reviews. Every governed item has an owner, status, version, effective date, review date, and linked evidence.

Policy Governance

Policy lifecycle states are draft, in_review, approved, scheduled, effective, superseded, and retired. Publication requires schema validation, test scenarios, impact analysis, approver identity, and effective time. Emergency changes are supported but remain versioned and require retrospective review.

Policy simulation runs proposed rules against historical or shadow traffic to estimate affected orders, alerts, providers, and users before activation. A policy change can be canaried by tenant, environment, account, strategy, or percentage of eligible traffic.

Audit Model

Audit evidence is append-only and searchable by tenant, actor, action, resource, time, correlation, policy, integration, agent, deployment, or incident. Sensitive payloads are referenced through protected snapshots rather than duplicated in plaintext. Hash chains or signed manifests may be used for high-assurance evidence packages.

Audit records cover:

- Authentication, membership, role, permission, and support access.
- Policy, playbook, prompt, model, agent, and connector changes.
- Risk evaluations, warnings, approvals, overrides, and kill switches.
- Order, execution, reconciliation, and account actions.
- Provider, quorum, route, incident, and runbook decisions.
- Data export, deletion, retention, and legal-hold actions.
- Deployments, migrations, feature-policy changes, and rollback.

Financial and Operational Recordkeeping

The system preserves the sequence of intent, data, risk decision, approval, order command, venue response, fill, reconciliation, review, and model analysis. Records use immutable versions or event history so that a later policy or model change does not alter the evidence for a prior decision.

Retention and report requirements are configurable by tenant and jurisdiction. Legal and compliance teams can define required fields, approval separation, export format, and retention without changing domain code.

Model and Agent Governance

Each model and agent has a model card or system card containing purpose, owner, data classes, expected use, prohibited use by policy, evaluation results, known failure modes, monitoring, and rollback. The platform supports unrestricted technical capability, but each deployment must explicitly authorize its use cases and autonomy level.

Model changes require comparison against the current production baseline. Evaluation results are segmented to expose regressions hidden by aggregate scores. Material degradation automatically blocks or rolls back a release according to policy.

Human Review and Approval

Review requirements are configurable by action type and risk. The platform supports single approval, multi-approval, sequential approval, quorum approval, separation of duties, and emergency break-glass. Approvers see the complete proposed action, evidence, policy results, and expiry. Approvals bind to the exact resource version; material changes invalidate them.

Evidence Packages

An evidence package is a signed or checksummed collection containing selected configuration versions, test results, audit records, deployment information, model evaluations, security scans, incident outcomes, and operational metrics. A manifest lists each artifact, its source, time range, checksum, and access classification.

Packages can be generated for production release, tenant onboarding, integration certification, incident review, security assessment, model change, policy change, or audit request.

Data and Integration Governance

Provider contracts, data rights, allowed storage, redistribution, retention, and model use are represented in provider configuration. The system can prevent an output from using a dataset outside its permitted purpose while retaining the adapter capability in the build. Data lineage allows downstream reports and model runs to identify all contributing sources.

Exception Management

Any exception to security, quality, risk, retention, or deployment policy requires a record with scope, owner, rationale, compensating controls, approval, start, expiry, and review. Expired exceptions automatically alert and may disable the affected configuration.

Training and Operational Readiness

Production launch includes role-specific training for traders, risk staff, operators, administrators, support, and incident responders. Training materials are versioned with the product. Critical workflows require completion or acknowledgment when tenant policy demands it.

Reliability, Observability, and Incident Management

Reliability Objectives

Reliability is defined by user and financial outcomes, not only server uptime. Service objectives distinguish read dashboards, transactional commands, execution, market data, provider quorum, AI analysis, jobs, and exports.

Reference production targets are design goals subject to environment and contract:

Service capability	Availability target	Latency or timeliness target
Authenticated read APIs	99.9 percent monthly	p95 under 500 ms for non-analytical queries
Transactional write APIs	99.9 percent monthly	p95 under 750 ms excluding external provider time
Execution command path	99.95 percent during supported market windows	Internal p95 under 250 ms before venue call
Realtime order updates	99.9 percent	p95 delivery under 2 seconds after receipt
Critical market-data freshness	Policy-specific	Alert before consumer freshness threshold is breached
Hermes quorum service	99.9 percent	p95 under configured provider timeout budget
Scheduled briefing	99.5 percent	Available by tenant-defined deadline
Background imports and analytics	99.0 percent completion	Progress visible; retries within job policy

Targets are measured per region and environment where applicable.

Observability Model

Every request, event, job, adapter call, provider probe, agent run, and database transaction carries correlation and trace context. Observability uses structured logs, metrics, distributed traces, error tracking, and domain audit events. Logs describe technical behavior; audit records describe accountable business actions. Neither substitutes for the other.

Required Metrics

Platform metrics

Request rate, error rate, latency, saturation, queue depth, worker concurrency, database connections, transaction duration, lock wait, cache hit, storage error, webhook delivery, and deployment health.

Trading metrics

Order submissions, acknowledgements, rejects, fills, partial fills, cancels, reconciliation differences, uncertain states, risk blocks, approvals, kill-switch events, and execution latency.

AI metrics

Run rate, schema validity, tool denial, tool failure, model fallback, latency, token usage, cost, evaluation score, user correction, and unsafe-action detection.

Hermes metrics

Provider availability, latency, freshness, divergence, quorum pass rate, no-decision rate, chain lag, reorg count, pool collection coverage, route failure, alert precision, and incident duration.

Logging Standards

Logs are JSON with timestamp, level, service, version, environment, tenant pseudonymous identifier where permitted, trace, correlation, event or job identifier, message, error code, and structured fields. Secrets, credentials, full payment data, raw auth tokens, and prohibited model content are redacted before emission.

Log volume and retention are controlled by classification. Sampling cannot remove errors, security events, financial commands, or incident evidence.

Alerting

Alerts are tied to symptoms, user impact, and runbooks. Each alert has owner, severity, threshold, evaluation window, deduplication key, escalation, and suppression policy. Alerts without an actionable response are converted to dashboards or removed.

Critical alerts include execution uncertainty, reconciliation drift, tenant-isolation failure, secret exposure, database unavailability, market-data staleness, provider quorum failure, chain reorganization beyond policy, and kill-switch activation.

Incident Lifecycle

Incident states are detected, triaged, active, mitigating, monitoring, resolved, and reviewed. Incidents record commander, severity, affected tenants and services, timeline, communications, actions, decisions, evidence, and customer impact. The platform supports internal and tenant-visible status updates with redaction controls.

Post-incident review identifies root causes, contributing factors, detection gaps, response gaps, corrective actions, owners, and due dates. Corrective actions become tracked work items linked to the incident.

Resilience Patterns

- Timeouts and bounded retries with jitter.
- Circuit breakers and bulkheads for providers and adapters.
- Idempotent consumers and commands.
- Transactional outbox and inbox deduplication.
- Dead-letter queues with replay tooling.
- Graceful degradation and explicit stale or unavailable states.
- Read-only or cancel-only modes during execution degradation.
- Provider failover and quorum.
- Database connection pooling and backpressure.
- Worker leases, heartbeat, and abandoned-job recovery.

Backup and Disaster Recovery

The production database uses automated backups and point-in-time recovery. Object storage uses versioning or equivalent protection according to classification. Configuration, secrets metadata, infrastructure state, and evidence manifests are backed up through separate mechanisms.

Reference recovery targets are:

- Transactional database RPO of five minutes or better.
- Critical configuration RPO of fifteen minutes or better.
- Core platform RTO of sixty minutes.
- Execution and risk command path RTO of thirty minutes where the deployment contract requires it.

Restore tests run on a schedule and produce evidence. A backup is not considered valid until restored and verified.

Capacity and Performance Engineering

Load models cover active users, concurrent websocket sessions, orders per second, market-data messages, provider probes, AI concurrency, backtests, import volume, storage growth, and export size. Capacity tests include burst conditions such as market open, high volatility, provider outage, chain congestion, and mass reconnection.

The system applies admission control and priority queues. Execution, risk, security, and incident work outrank bulk backtests or non-urgent AI analysis during contention.

Status and Transparency

An internal service catalog lists owners, dependencies, SLOs, dashboards, runbooks, and current health. Tenant-facing status surfaces relevant incidents and integration health without exposing other tenants or sensitive infrastructure detail.

Testing, Evaluation, and Quality Engineering

Quality Strategy

Quality is built through layered automated tests, realistic environments, production-like evidence, and controlled release. Test completion includes behavior, security, tenant isolation, observability, recovery, accessibility, and documentation.

Test Pyramid and Suites

Test type	Scope
Unit	Pure calculations, value objects, parsers, state transitions, policy expressions
Property-based	Financial invariants, rounding, order state machines, idempotency, serialization
Component	Domain services with real repositories or isolated external adapters
Integration	Postgres, RLS, storage, cache, event bus, secrets, provider sandboxes
Contract	API schemas, events, webhooks, broker, market-data, model, and chain adapters
End-to-end	Full user and operator workflows through UI and APIs
Simulation	Market, execution, provider, chain, and failure scenarios
Performance	Load, spike, soak, queue, database, websocket, and analytical workloads
Security	Authentication, authorization, tenant isolation, abuse, injection, secrets, supply chain
AI evaluation	Factuality, schema, tools, policy, calibration, regressions, adversarial cases
Accessibility	Automated and manual keyboard, screen-reader, contrast, and chart alternatives
Recovery	Backup restore, failover, replay, reconciliation, rollback, and disaster exercises

Deterministic Financial Tests

Risk, accounting, sizing, fee, currency, position, and portfolio calculations use golden cases and property tests. Invariants include conservation of quantity and cash where applicable, no duplicate fill application, stable rounding, reproducible policy decisions, and consistency between event replay and current projection.

RLS and Authorization Tests

RLS tests create multiple tenants, users, roles, resources, and service identities against a real Postgres instance. They attempt permitted and prohibited operations using actual tokens or session claims. Tests cover direct table access, views, functions, storage, realtime, vectors, and background workers.

Every new tenant-owned table must include a failing cross-tenant test before its migration is accepted.

Execution Tests

Execution testing uses simulators, provider sandboxes, recorded fixtures, and controlled paper accounts. Scenarios include duplicate submission, timeout after venue acceptance, partial fill, reject, cancel race, replace race, disconnected stream, stale account, market halt, insufficient buying power, reconciliation difference, and adapter failover.

Live certification begins with restricted canary accounts and low exposure defined by deployment policy. The code supports full scale, while rollout evidence controls operational progression.

Provider and Hermes Tests

Provider tests validate schema, timeouts, rate limits, pagination, malformed data, staleness, divergence, and recovery. Quorum tests cover missing providers, outliers, correlated failure, stale-majority conditions, weighting changes, and no-decision. Chain tests cover reorgs, finality delay, token decimals, event duplication, block gaps, contract upgrades, and transaction simulation failures.

AI Evaluation Harness

Evaluation datasets contain task input, approved evidence, expected structured properties, rubric, and prohibited errors. Suites include normal, edge, adversarial, low-data, conflicting-evidence, stale-data, and policy-denied cases. Results compare candidate and production versions with confidence intervals where feasible.

Tool-enabled agents are tested for correct tool choice, valid arguments, permission adherence, idempotency, refusal to use unavailable data, and behavior after tool failure. Financial actions are tested in simulation and paper environments before live authorization.

End-to-End Critical Paths

Required E2E paths include:

1. Tenant creation, membership, role assignment, and denied cross-tenant access.
2. Broker connection, account discovery, import, normalization, and reconciliation.
3. Trade intent, risk evaluation, approval, paper order, fill, journal, and analysis.
4. Live-manual order canary with audit and reconciliation in an approved environment.
5. Automated strategy order in simulation and paper modes, including kill switch.
6. Briefing generation with provider freshness and evidence drill-down.
7. Backtest submission, progress, reproducibility manifest, and comparison.
8. Hermes provider degradation, quorum fallback, incident, runbook, and recovery.
9. Policy publication, simulation, canary, activation, and rollback.
10. Evidence-package generation from deployment, tests, audit, and metrics.

Test Data

Synthetic data is preferred for general tests. Sanitized or tenant-approved production-like data can be used under restricted controls. Test factories create realistic instruments, accounts, orders, fills, chain observations, and provider failures. Time and random generators are injectable for deterministic replay.

Quality Gates

A release candidate must pass:

- Type checking, linting, formatting, and dependency boundaries.
- Unit, integration, RLS, contract, E2E, and migration tests.
- Codex HITL policy, approval-token, workflow-state, shell-free executor, output-bound, timeout, SQLite audit, and ten-test baseline checks.
- Automated requirement-traceability validation with no unresolved mandatory links.

- Security and secret scans with no unaccepted critical findings.
- AI evaluation thresholds for changed agents or models.
- Performance regression thresholds for changed critical paths.
- Accessibility checks for changed user workflows.
- Rollback, migration, and observability verification.
- Evidence manifest generation.

Exceptions are governed, time-limited, and visible in release evidence.

Coverage and Mutation Targets

Line coverage alone is insufficient. Critical deterministic packages should target at least 90 percent statement coverage and meaningful branch coverage, with mutation testing on risk, order state, reconciliation, and policy code. Lower-risk UI and integration glue use risk-based thresholds supplemented by E2E tests.

Production Verification

After deployment, automated smoke tests validate authentication, tenant context, database access, key APIs, workers, realtime delivery, market-data freshness, provider quorum, agent execution, and environment-specific execution mode. Canary metrics are compared to baseline before full rollout.

Repository and build verification also runs `codex-hitl verify` against the exact release commit, confirms the ten-test baseline, validates traceability, checks policy and lockfile hashes, and attaches the resulting evidence to the release record.

DevOps, Environments, Deployment, and Release Management

Environment Model

The platform uses local, CI, preview, development, staging, and production environments. Tenants also have logical sandbox, paper, and production contexts. Environment configuration is schema-validated and versioned. Production data and secrets do not flow to lower environments by default.

Infrastructure as Code

Networking, compute, database configuration, queues, cache, storage, secrets, observability, edge controls, DNS, and access policies are defined as code. Plans are reviewed and policy-checked before application. Manual emergency changes are detected through drift monitoring and reconciled into source control.

CI Pipeline

Pull-request CI performs dependency installation with locked versions, generated-code verification, lint, type check, unit tests, affected integration tests, migration checks, RLS tests, security scans, container build, SBOM generation, and preview deployment where applicable.

The Codex control-plane job additionally validates Spec Kit artifacts, loads and tests the TOML policy, runs the ten HITL baseline cases, executes `codex-hitl verify`, confirms requirement traceability, performs the non-root container smoke test, and publishes commit-bound evidence.

Main-branch CI adds full integration and E2E suites, signed artifacts, provenance attestations, and release-candidate publication. Production deployment uses only artifacts built by CI.

Database Migrations

Migrations are forward-only, ordered, reviewed, and tested from empty and production-like schemas. Expand-and-contract patterns preserve backward compatibility across rolling deployments. Destructive changes require archival, backfill, verification, and delayed cleanup. RLS policies and grants are reviewed as part of every migration.

Release Strategies

The system supports rolling, blue-green, and canary deployment. Feature policies can expose a release by tenant, user, environment, account, provider, chain, or percentage. Critical execution changes use a shadow or paper phase, canary accounts, and enhanced monitoring before broad activation.

Configuration Management

Configuration values are divided into code defaults, environment configuration, tenant configuration, versioned policy, and secrets. Runtime configuration is validated and observable. Changes to high-impact settings create audit events and may require approval.

No product capability is removed from the codebase through environment-specific forks. Exposure is controlled through capability and policy manifests.

Artifact and Supply-Chain Integrity

Builds produce signed containers or deployable bundles, checksums, SBOMs, dependency manifests, and source provenance. Base images and dependencies are pinned and scanned. Deployment verifies signatures. Administrative access to build and deployment systems uses strong authentication and least privilege.

Rollback and Forward Recovery

Every release documents application rollback, configuration rollback, feature-policy disablement, worker pause, and database forward-recovery procedures. A migration that cannot be rolled back must have a tested forward fix and a compatibility window.

Scheduled Jobs and Backfills

Schedulers use tenant-aware jobs with unique keys, leases, retries, dependency state, and progress. Backfills run with bounded concurrency and priority below critical operations. A backfill is versioned and records query range, code version, output counts, errors, and reconciliation.

Cost and Resource Governance

Usage is measured by API, storage, market data, AI, computation, backtest, provider, and event volume. Budget alerts and quotas are configurable, but the architecture does not hard-code product caps. Expensive workloads can be scheduled, prioritized, or routed to cost-appropriate resources without changing domain behavior.

Operational Documentation

Each service and capability includes architecture, ownership, runbook, dashboards, alerts, SLO, dependencies, deployment steps, rollback, test evidence, and support guidance. Documentation is versioned with code and checked for broken references.

The repository additionally maintains SECURITY.md, the Codex HITL threat model, approval and policy operations, SQLite backup and recovery, token-key rotation, stuck-workflow recovery, traceability remediation, and verification-evidence procedures.

Delivery Program, Workstreams, and Phase Plan

Delivery Strategy

The program delivers in fourteen phases aligned with the original charter. Phases control implementation order and evidence, not final product scope. Domain entities, contracts, policy, tenant security, event envelopes, and observability are designed for the complete platform from the beginning.

Ten workstreams operate in parallel: product architecture, backend infrastructure, AI and ML, risk and analytics, market data, frontend and UX, security and RBAC, testing and QA, DevOps and production, and compliance and operations.

Phase 1 - Platform Foundation

Deliverables: monorepo, CI, local environments, Supabase schema, tenant and membership model, RLS test harness, RBAC and ABAC foundation, audit events, configuration, web shell, API skeleton, event envelope, outbox, observability, and architecture decision records.

Exit evidence: successful tenant-isolation tests; automated deployment to development; health, logs, and traces; migration verification; authentication and role workflows; baseline threat model.

Phase 2 - Trade Journal MVP

Deliverables: manual entry, CSV import, broker import framework, instruments, accounts, orders, executions, trades, notes, tags, screenshots, reconciliation, core metrics, and journal UI.

Exit evidence: reproducible import; duplicate and timezone tests; full journal E2E; audit and source lineage; accessibility review.

Phase 3 - Risk Manager MVP

Deliverables: policy model, sizing, risk evaluation, daily loss, drawdown, exposure, concentration, warnings, approvals, breaches, and risk console.

Exit evidence: golden calculation suite, policy simulation, override audit, pre-trade E2E, performance baseline.

Phase 4 - Market Open Briefing MVP

Deliverables: market provider framework, calendars, reference data, quotes and bars, freshness, provider health, briefing scheduler, watchlists, briefing agent, and evidence drill-down.

Exit evidence: provider contract tests, deadline SLO, stale-data scenarios, model evaluation, tenant configuration.

Phase 5 - AI Trading Coach

Deliverables: agent registry, prompt versions, tool framework, retrieval, memory, coaching observations, improvement plans, weekly review, feedback, and evaluation harness.

Exit evidence: labeled evaluation suite, structured-output validity, privacy tests, user review workflow, cost and latency dashboard.

Phase 6 - Hermes Durable Backend

Deliverables: chains, networks, providers, endpoints, tokens, pools, observations, collector workers, health samples, and durable repositories.

Exit evidence: migration and retention tests, provider fixtures, chain-context validation, replayable collections.

Phase 7 - Hermes RBAC and Tenant Isolation

Deliverables: operator roles, scoped permissions, privileged actions, storage policy, tenant-aware jobs, provider ownership, and support access controls.

Exit evidence: real RLS and ABAC tests, cross-tenant attack suite, audit search, break-glass test.

Phase 8 - Provider Adapters and Quorum

Deliverables: multiple provider adapters, scoring, quorum policies, canonical decisions, divergence, no-decision behavior, and provider console.

Exit evidence: correlated-failure simulations, stale-majority tests, quorum accuracy and latency metrics, failover runbook.

Phase 9 - Hermes Operator Dashboard

Deliverables: health, chain, pool, quorum, alert, incident, and runbook interfaces; realtime updates; evidence package generation.

Exit evidence: incident E2E, accessibility, operator training, status and notification tests.

Phase 10 - Strategy and Advanced Analytics

Deliverables: playbooks, portfolio analytics, backtesting, scenarios, probability, regime detection, strategy discovery, and research workspace.

Exit evidence: reproducibility, look-ahead tests, calibration reports, performance and scale tests.

Phase 11 - Screenshot and Replay Intelligence

Deliverables: vision pipeline, chart extraction, screenshot annotations, market replay, decision timelines, and counterfactual review tools.

Exit evidence: image evaluation set, confidence and confirmation flows, file security, replay integrity tests.

Phase 12 - Broker and Exchange Execution

Deliverables: account discovery, order state machine, execution gateway, simulation, paper, manual live, supervised live, automated live, reconciliation, routing, and execution analytics.

Exit evidence: adapter certification, duplicate and timeout tests, kill-switch exercises, paper soak, live canary under approved policy, reconciliation thresholds.

Phase 13 - Production Hardening

Deliverables: SLOs, capacity, autoscaling, backup and restore, disaster recovery, security testing, penetration remediation, cost controls, support tools, multi-region options, and operational documentation.

Exit evidence: load and soak reports, restored backups, incident exercises, security sign-off, production dashboards, on-call readiness.

Phase 14 - Legal, Compliance, Training, and Sign-Off

Deliverables: tenant terms and configuration support, recordkeeping settings, data and provider rights register, model cards, training, final evidence package, launch checklist, and approval record.

Exit evidence: required organizational sign-offs, completed training, current evidence, no unresolved release blockers, rollback readiness.

Parallel Workstream Responsibilities

Workstream	Primary accountability
Product architecture	Scope, domain boundaries, ADRs, requirements, acceptance
Backend infrastructure	APIs, repositories, events, jobs, integrations, performance
AI and ML	Agents, models, retrieval, evaluations, vision, calibration
Risk and analytics	Policy, sizing, portfolio, metrics, backtesting, scenarios
Market data	Reference data, normalization, providers, freshness, calendars
Frontend and UX	Design system, workflows, accessibility, realtime interaction
Security and RBAC	Identity, authorization, RLS, secrets, threat models, testing
Testing and QA	Test strategy, automation, E2E, performance, evidence
DevOps and production	CI/CD, infrastructure, observability, SRE, recovery
Compliance and operations	Governance, records, incidents, training, launch evidence

Program Controls

Each phase has a definition of ready, architecture review, threat-model update, data review, test plan, observability plan, documentation plan, and exit review. Work can proceed in parallel when contracts are stable. A phase is not complete because the UI works; it is complete when data, APIs, security, tests, operations, and evidence are accepted.

Metrics, Acceptance Criteria, and Readiness Gates

Trader Outcome Metrics

- Change in expectancy and average R-multiple by comparable strategy and regime.
- Reduction in risk-policy violations, revenge trades, oversizing, and unplanned entries.
- Improvement in journal completion, plan completion, and review completion.
- Process-grade improvement independent of profit and loss.
- Reduction in avoidable execution cost and late or missed exits.
- Coaching goal completion and recurrence rate of targeted behaviors.

Metrics are interpreted with sample size, market conditions, and selection effects; the product does not claim causation from uncontrolled aggregates.

Product Metrics

- Daily and weekly active tenants and users.
- Connected accounts, imported trades, reconciled records, and journal completion.
- Briefing open and evidence drill-down rates.
- Watchlist use, trade-intent creation, risk-warning acknowledgement, and approval timing.
- Agent use, correction, acceptance, and repeat use.
- Backtests, research projects, reports, and exports.
- Conversion between observe, simulation, paper, and live execution modes where enabled.

Risk and Execution Metrics

- Risk evaluation latency and decision distribution.
- Breach count, severity, duration, recurrence, and remediation.
- Approval volume, response time, expiry, and override rate.

- Order acknowledgement, reject, fill, cancel, and reconciliation rates.
- Implementation shortfall, slippage, spread, fill rate, and venue comparison.
- Kill-switch activations, false positives, time to containment, and recovery.

AI Quality Metrics

- Structured-output validity.
- Evidence coverage and unsupported-claim rate.
- Tool selection and argument correctness.
- User correction and rejection rate.
- Confidence calibration.
- Policy and permission violation rate.
- Model fallback, latency, token, and cost.
- Evaluation-suite pass rate by agent version and segment.
- Downstream outcome metrics where attribution is defensible.

Hermes Metrics

- Provider availability, freshness, latency, error, and coverage.
- Quorum pass, degraded, and no-decision rates.
- Divergence detection and alert precision.
- Chain lag, reorganization handling, and backfill completion.
- Pool and route coverage, quote success, simulation success, and route expiry.
- Incident detection time, acknowledgement time, mitigation time, and recurrence.
- Runbook success, rollback, and operator intervention rates.

Production Metrics

- Deployment frequency, lead time, change failure rate, and recovery time.
- SLO attainment and error-budget consumption.
- Test pass, flaky-test, and escaped-defect rates.
- Security finding age and remediation.
- Backup success and restore verification.
- Queue age, job success, webhook success, and data reconciliation.
- Cost per active tenant, trade, agent run, provider observation, and backtest.

Capability Acceptance Template

Every capability is accepted against:

1. Approved requirements and user journeys.
2. Domain model and API contracts.
3. Authorization and tenant isolation.
4. Deterministic calculations or model evaluation as applicable.
5. Error, retry, idempotency, and reconciliation behavior.
6. Logs, metrics, traces, dashboards, alerts, and runbooks.
7. Accessibility and responsive behavior.
8. Documentation and support guidance.
9. Performance and capacity threshold.
10. Deployment, rollback, and evidence package.

Production Readiness Gates

A production release requires all of the following:

- No unresolved critical security or tenant-isolation defect.
- RLS and authorization suites pass against the production-equivalent schema.
- Critical E2E paths pass in staging with production-like integrations.
- Changed agents and models meet evaluation thresholds.
- Migration, rollback, and backup restore are verified.
- SLO dashboards, alerts, and runbooks are active.
- Incident ownership and on-call coverage are established.
- Data, provider, and model configurations are approved.
- Execution modes and limits are explicitly configured per tenant and environment.
- Evidence package is generated, checksummed, and approved.

Release Decision

Release status is one of `go`, `go_with_time_limited_exception`, or `no_go`. Exceptions cannot cover critical tenant isolation, secret exposure, unreconciled financial action, or inability to stop execution. Other exceptions require owner, expiry, compensating controls, and executive or designated approval.

Final Definition of Done and Build Baseline

Final Build Definition

The build is complete when the platform implements the shared foundation, all Track A and Track B capability domains, the complete execution-mode architecture, tenant and policy controls, production operations, and evidence required by this paper. Delivery sequencing may defer a user-facing screen, but it may not require redesign of canonical identifiers, tenancy, policy, events, audit, or integration boundaries to add an already-defined capability.

The engineering baseline is also incomplete until the Spec Kit artifacts, AGENTS.md rules, deterministic TOML policy engine, approval tokens, workflow state machine, SQLite starter history, shell-free bounded executor, traceability validator, ten-test baseline, CI workflow, Dockerfile, security policy, runbook, and Git baseline commit satisfy the Codex HITL acceptance standard.

No-Arbitrary-Restriction Acceptance

The final architecture shall demonstrate that:

- All capability limits are represented as configuration, policy, permissions, external capability, or infrastructure capacity rather than undocumented hard-coded restrictions.
- The order system supports observe, simulation, paper, manual live, supervised live, and automated live modes through the same contracts.
- The integration framework can add brokers, exchanges, providers, chains, venues, and AI models without core-domain modification.
- Asset classes and instrument types are extensible through metadata and specialized modules.
- Agent autonomy can be increased or reduced through versioned policy and tool permissions.
- Tenant, environment, resource, and role scope can be configured without source-code forks.
- Data retention, provider quorum, risk thresholds, approvals, and operational limits are versioned and auditable.
- No commercial package, UI navigation decision, or temporary MVP choice becomes a permanent architecture limitation.

Mandatory Completion Evidence

The final release evidence includes:

- Architecture and data-model review.
- RLS and tenant-isolation results.
- RBAC and ABAC authorization matrix results.
- Risk and financial-calculation test results.
- Broker and provider adapter certification.
- Simulation, paper, and approved live-canary execution evidence.
- Agent and model evaluation reports.
- Hermes quorum, reorg, incident, and runbook tests.
- Security scans and penetration-test disposition.
- Performance, load, and soak reports.
- Backup restore and disaster-recovery exercise.
- Accessibility and critical UX verification.
- Deployment, rollback, configuration, and evidence manifests.
- Spec Kit constitution, specification, plan, tasks, and acceptance-check evidence.
- Codex AGENTS.md instruction-discovery evidence.
- TOML policy hash, deterministic policy test vectors, and approval-token test results.

- Workflow-state, SQLite append-only, shell-free executor, output-bound, and timeout evidence.
- Ten-test baseline and separate requirement-traceability PASS report.
- GitHub Actions run, non-root Docker smoke result, and verified Git commit.
- Training and operational sign-off.

Final Operational Principle

The system is built for broad capability and controlled operation. It does not reduce scope by hard-coding blanket prohibitions, and it does not confuse unrestricted engineering capability with uncontrolled production behavior. Every action remains attributable, authorized, policy-evaluated, observable, and recoverable.

The same distinction applies to Codex-assisted engineering: the repository retains full implementation scope, while each governed command and completion claim remains exact, reviewable, bounded, traceable, and tied to evidence from a specific commit.

Authority of This Paper

This paper is the authoritative consolidated product and engineering baseline for the Unified AI Trading Operating System and Hermes Protocol. Phase specifications, stories, code, schemas, tests, infrastructure, and operational procedures shall trace to it. Deviations require an approved architecture decision record and corresponding updates to requirements, tests, and evidence.

Appendix A - Reference Monorepo

```
repository-root/  
  AGENTS.md  
  SECURITY.md  
  Dockerfile  
  .specify/  
    memory/  
      constitution.md  
  specs/  
    000-codex-hitl-control-plane/  
    001-platform-foundation/  
    ...  
  policies/  
    default.toml  
    schema.json  
  apps/  
    web/  
    hermes-ops/  
    platform-admin/  
    api/  
    realtime/  
    webhooks/  
    workers/  
    quant-workers/  
    scheduler/  
    connector-runtime/  
    docs/  
  packages/  
    core/  
    identity/  
    policy/  
    trading/  
    portfolio/  
    risk/  
    journal/  
    analytics/  
    market-data/  
    ai/  
    hermes/  
    integrations/  
    events/  
    observability/  
    ui/  
    testing/  
    config/  
  tooling/  
    codex-hitl/  
      pyproject.toml  
      src/codex_hitl/  
      tests/  
  supabase/  
    migrations/  
    seed/  
    tests/  
    config.toml  
  infra/  
    modules/  
    environments/  
    policies/  
  docs/  
    architecture/  
    adr/  
    api/  
    runbooks/
```

```
security/
threat-models/
evidence/
scripts/
  validate_traceability.py
evidence/
.github/
  workflows/
    ci.yml
package.json
pnpm-workspace.yaml
turbo.json
pyproject.toml
README.md
```

Appendix B - Representative Permission Matrix

Permission	Trader	Coach	Risk manager	Hermes operator	Tenant admin
trade.read	Yes	Scoped	Yes	No	Yes
trade.write	Yes	No	Scoped	No	Yes
order.submit.paper	Policy	No	Policy	No	Policy
order.submit.live	Policy	No	Policy	No	Policy
order.submit.automated	Delegated	No	Approve	No	Configure
risk.evaluate	Yes	Read	Yes	No	Yes
risk.policy.publish	No	No	Yes	No	Yes
risk.override	No or policy	No	Policy	No	Policy
coach.review	Own	Yes	Read	No	Yes
agent.run	Policy	Policy	Policy	Policy	Configure
provider.read	No	No	Read	Yes	Yes
provider.configure	No	No	No	Policy	Yes
incident.manage	No	No	Scoped	Yes	Yes
audit.read	Own	Scoped	Yes	Yes	Yes
member.manage	No	No	No	No	Yes

Actual decisions also apply ABAC conditions for tenant, environment, account, resource, time, authentication strength, and approval state.

Appendix C - Representative Event Catalog

Event	Producer	Principal consumers
tenant.member_added.v1	Identity	Notifications, audit projections, access cache
policy.published.v1	Policy	Risk, execution, agents, UI capability cache
import.completed.v1	Integrations	Trading, journal, analytics, notifications
trade.closed.v1	Trading	Journal analysis, coaching, metrics
risk.evaluated.v1	Risk	Execution, approvals, UI, audit
risk.breach_opened.v1	Risk	Notifications, incident workflows, dashboards

Event	Producer	Principal consumers
order.submitted.v1	Execution	Realtime, audit, reconciliation
order.filled.v1	Execution	Portfolio, journal, risk, analytics
reconciliation.difference_found.v1	Trading	Operations, risk, notifications
briefing.generated.v1	Market intelligence	Notifications, UI, analytics
agent.run_completed.v1	AI	UI, feedback, analytics, downstream workflows
provider.health_changed.v1	Hermes	Quorum, incidents, dashboards
quorum.no_decision.v1	Hermes	Risk, execution, incidents, alerts
chain.reorg_detected.v1	Hermes	Recompute, incidents, route invalidation
incident.opened.v1	Operations	Notifications, status, runbooks
deployment.completed.v1	DevOps	Audit, evidence, smoke tests

Appendix D - Risk Policy Example

```
{
  "policy_set": "Primary Trading Policy",
  "scope": {
    "environment": "production",
    "portfolio_id": "por_..."
  },
  "version": 12,
  "rules": [
    {
      "code": "MAX_TRADE_RISK",
      "when": {"instrument.asset_class": {"in": ["equity", "future", "digital_asset"]}},
      "measure": "trade.loss_at_invalidation_pct_equity",
      "operator": "lte",
      "limit": 0.005,
      "on_fail": "require_approval",
      "override": {"allowed": true, "permission": "risk.override", "max_duration_minutes": 30}
    },
    {
      "code": "DAILY_LOSS_STOP",
      "measure": "portfolio.realized_and_unrealized_daily_loss_pct",
      "operator": "gt",
      "limit": 0.02,
      "on_fail": "block",
      "actions": ["activate_kill_switch:new_exposure"]
    },
    {
      "code": "MARKET_DATA_CONFIDENCE",
      "measure": "market.reference_price_confidence",
      "operator": "gte",
      "limit": 0.90,
      "on_fail": "block"
    }
  ]
}
```

Appendix E - Agent Release Checklist

- Purpose, owner, and data classification are documented.
- Input and output schemas are versioned.
- Tool allowlist and side-effect levels are approved.
- Model routing and fallback behavior are tested.

- Retrieval collections and authorization are verified.
- Offline evaluation meets thresholds by segment.
- Adversarial prompt and tool tests pass.
- Cost, latency, error, and fallback dashboards exist.
- Rollback to the prior version is tested.
- Human-review or autonomy policy is explicitly configured.
- Audit and feedback records are generated.
- Production canary and monitoring plan are approved.

Appendix F - Production Readiness Checklist

Product and Workflow

- Critical user journeys are complete and documented.
- Errors and recovery paths are actionable.
- All material outputs show provenance and freshness.
- Role-specific training is available.

Data and Security

- Migrations, RLS, authorization, storage, and vectors are tested.
- Secrets are vaulted and rotation is verified.
- Data retention, export, deletion, and legal-hold policies are configured.
- Threat models and security findings are current.

Trading and Risk

- Risk calculations and policy cases pass.
- Order state, idempotency, adapter, and reconciliation cases pass.
- Kill switches and uncertain-state recovery are exercised.
- Execution modes are explicitly configured per tenant and account.

AI and Analytics

- Agent and model evaluations pass.
- Backtests and analytics are reproducible.
- Confidence, uncertainty, and evidence are visible.
- Model fallback and rollback are tested.

Hermes and Providers

- Provider adapters and health probes pass contract tests.
- Quorum and no-decision scenarios pass.
- Reorg, stale data, divergence, and incident scenarios pass.
- Operator runbooks and evidence packages are verified.

Operations

- SLOs, dashboards, alerts, and on-call ownership are active.
- Capacity, load, and soak tests meet thresholds.
- Backup restore and disaster recovery are demonstrated.
- Deployment, canary, rollback, and smoke tests are verified.
- Codex HITL verification, ten-test baseline, traceability, CI, Docker smoke, and commit-bound evidence pass.
- Security policy, threat model, approval operations, and recovery runbooks are current.

Appendix G - Glossary

Term	Definition
ABAC	Attribute-based access control using resource and contextual conditions
ADR	Architecture decision record
Agent	Versioned AI workflow with instructions, models, tools, schemas, policy, and evaluation
Canonical record	Authoritative normalized domain representation independent of a vendor payload
Capability manifest	Signed description of modules and actions available to a tenant session
Correlation ID	Identifier linking requests, events, jobs, tool calls, and audit evidence
Execution mode	Observe, simulate, paper, live manual, live supervised, or live automated operation
Hermes	Multi-chain provider, liquidity, quorum, route, and operator subsystem
Idempotency	Property that safe retries do not create duplicate effects
Kill switch	Policy-controlled stop state for new exposure or other high-impact actions
Quorum	Confidence-rated canonical decision derived from multiple eligible provider observations
RLS	Postgres row-level security
R-multiple	Trade result expressed relative to planned or realized risk unit
Structured output	Schema-validated agent result used as the authoritative machine representation
Tenant	Isolated customer or internal workspace with its own membership, policy, data, and environment
Transactional outbox	Pattern that writes state and publishable events in one database transaction
Approval token	High-entropy, hashed, expiring, request-scoped, single-use authorization credential
Codex HITL	Human-in-the-loop engineering control plane for repository policy, approval, command execution, audit, and verification
Policy hash	Cryptographic digest identifying the exact immutable policy used for a decision
Requirement traceability	Verified links from normative requirements through tasks, code, tests, checklists, evidence, and release commit

Appendix H - Codex HITL Requirement-to-Evidence Matrix

Requirement	Principal implementation	Mandatory evidence
HITL-001	.specify, specs, checklists	Traceability report with no missing links
HITL-002	Root and scoped AGENTS.md	Instruction-discovery output
HITL-003 to HITL-004	policy.py, TOML schema and policies	Deterministic policy tests and policy hash

Requirement	Principal implementation	Mandatory evidence
HITL-005	approvals.py, token tables	Expiry, replay, mismatch, and concurrency tests
HITL-006	workflow.py, transition table	Transition coverage and invalid-transition test
HITL-007	audit.py, SQLite migrations	Restart durability and append-only verification
HITL-008 to HITL-009	executor.py	Shell-free invocation, bounds, timeout, and process cleanup tests
HITL-010	traceability.py, validation script	JSON and readable traceability report
HITL-011	Test suite	Ten mapped baseline results plus project tests
HITL-012	.github/workflows/ci.yml	Successful protected-branch CI run
HITL-013	Dockerfile	Image digest and offline smoke result
HITL-014	SECURITY.md, threat model, runbook	Documentation checklist and owner approval
HITL-015	Git repository	Commit identifier and clean-state evidence
HITL-016 to HITL-018	CLI, verification report, adapters	codex-hitl verify report and enterprise gap register

Appendix I - Approval and Execution Data Contracts

Canonical action request

```
{
  "request_id": "req_01...",
  "correlation_id": "cor_01...",
  "requested_by": "usr_01...",
  "repository_id": "repo_ai_trading_os",
  "commit": "0123456789abcdef...",
  "working_directory": "tooling/codex-hitl",
  "action": "run_verification",
  "argv": ["codex-hitl", "verify", "--json", "evidence/verification.json"],
  "environment_names": ["PATH", "HOME"],
  "network": false,
  "workspace_only": true,
  "timeout_seconds": 120,
  "stdout_limit_bytes": 1048576,
  "stderr_limit_bytes": 1048576,
  "side_effect": "verification",
  "requirement_ids": ["HITL-010", "HITL-016"]
}
```

The request hash is computed from a canonical serialized representation with stable field ordering, normalized paths, explicit defaults, and no secret values.

Approval record

```
{
  "approval_request_id": "apr_01...",
  "request_sha256": "...",
  "policy_sha256": "...",
  "status": "granted",
  "requested_by": "usr_01...",
  "approved_by": "usr_02...",
  "reason": "Verified migration and rollback plan",
  "issued_at": "2026-07-10T16:00:00Z",
}
```

```

"expires_at": "2026-07-10T16:15:00Z",
"max_uses": 1,
"uses": 0,
"token_verifier": "redacted"
}

```

Execution result

```

{
  "execution_id": "exe_01...",
  "workflow_id": "wfl_01...",
  "request_sha256": "...",
  "started_at": "2026-07-10T16:01:00Z",
  "finished_at": "2026-07-10T16:01:08Z",
  "duration_ms": 8034,
  "exit_code": 0,
  "timed_out": false,
  "canceled": false,
  "stdout_bytes": 18422,
  "stderr_bytes": 0,
  "stdout_truncated": false,
  "stderr_truncated": false,
  "stdout_sha256": "...",
  "stderr_sha256": "...",
  "post_checks_passed": true
}

```

Appendix J - Starter Completion Report Template

The following report format may be emitted only from a passing verification record:

```

Starter implementation complete

Commit: <verified commit>
Policy: <version and SHA-256>
Database schema: <migration version>

Implemented:
- Spec Kit-compatible constitution, specification, plan, tasks, and acceptance checklist
- Codex AGENTS.md operating rules
- deterministic TOML policy engine
- allow / require-approval / deny decisions
- expiring, hashed, single-use approval tokens
- validated workflow state machine
- SQLite audit and execution history
- command execution using shell=False
- bounded stdout/stderr capture and timeouts
- automated requirement-traceability validation
- unit tests, GitHub Actions CI, Dockerfile, security policy, and operations runbook
- initialized Git repository with a baseline commit

Verification:
- Tests: <passed>/<collected> passed; <failed> failed; <skipped> skipped
- Ten-test baseline: PASS
- Traceability: PASS
- Container smoke: PASS
- Worktree: CLEAN or documented exception

Enterprise production gaps:
- <identity, central audit, secrets, policy signing, deployment sandboxing, or none>

```

The completion report shall include links or artifact identifiers for detailed test output, traceability, audit export, container image, and CI run. It shall never conceal skipped checks or known production gaps.

Appendix K - External Reference Sources

Source	Relevance	Publication-time note
OpenAI Codex - Agent approvals and security	Sandbox, approval, and network operating guidance	Verified July 10, 2026
OpenAI Codex - Sandbox	Workspace-write and approval configuration	Verified July 10, 2026
OpenAI Codex - Custom instructions with AGENTS.md	Repository instruction discovery and precedence	Verified July 10, 2026
OpenAI Codex - Managed configuration	Organization policy for sandbox, approval, network, and tools	Verified July 10, 2026
GitHub Spec Kit - README	Specification-driven project workflow and supported installation model	Official repository
GitHub Spec Kit - Releases	Reviewed release pin	v0.12.9 current when verified

End of Final Production Paper